

DAY 4 PostgreSQL 18 Backup and Disaster Recovery

Complete step by step lab manual

Consolidated edition with explicit kubectl commands | 29 September 2026

This manual brings together the seven supplied manuals and retains explicit kubectl commands throughout. Follow Labs 0 through 9 in order to back up the live Zalando and Patroni cluster, archive WAL to GCS, and recover onto independent storage. Appendix A retains the original disposable backup-lab exercises. Appendix B records how the earlier versions were reconciled so setup and recovery steps can be checked explicitly.

The commands use Bash on Ubuntu or WSL. Multiline commands are intentional. Copy an entire code block, including its final EOF marker. A backslash must be the last character on its line. Do not paste Bash commands directly into Windows CMD or PowerShell. No database or cloud commands in this manual have been executed against your environment during preparation; each checkpoint must be verified in your lab.

Physical restore and PITR use new PVCs and standalone PostgreSQL processes. They do not mount live Patroni storage, join the live Patroni scope, or archive recovered timelines back into the live repository. This is a recovery drill; production cutover and rebuilding an HA cluster require a separate change plan.

How to use the manual

Run workstation blocks from one Bash terminal in the day4-complete directory. Each kubectl exec command shows its target pod and container explicitly. Where required, runuser selects the PostgreSQL operating-system user. SQL blocks are passed through psql with ON_ERROR_STOP. There are no custom shell command shortcuts to learn. File-creation blocks include their complete contents.

Every lab states its objective, prerequisites, commands, expected result, and stop conditions. A successful command exit is necessary but the expected data and recovery state must also agree. Keep the companion text file open when copying long manifests from a PDF reader.

Contents

Lab 0 Workstation and cluster validation	4
Lab 1 Logical backup and restore	8
Lab 2 Physical backup using pg_basebackup	10
Lab 3 Inspect existing WAL archiving	13
Lab 4 Configure pgBackRest completely	15
Lab 5 Full backup	27
Lab 6 Incremental backup	29
Lab 7 Differential backup and retention	31
Lab 8 Complete physical restore to independent storage	34
Lab 9 Complete point in time recovery	41
Operations and troubleshooting	47
Appendix A Disposable backup lab	53
Appendix B Coverage and corrections	62
References and verification scope	64

Environment and naming

Namespace: postgres-ha

Live PostgreSQL custom resource: acid-postgres-ha

Live primary Service: acid-postgres-ha

Live container: postgres

Expected live members: acid-postgres-ha-0, -1 and -2

PostgreSQL major version: 18

pgBackRest stanza for the live cluster: postgres-ha

Configuration: /etc/pgbackrest/pgbackrest.conf

GCS bucket used in supplied manuals: postgres-ha-backups-parameswari-2026

Repository prefix: /pgbackrest/postgres-ha

Cipher Secret: pgbackrest-repo-secret, key repo-cipher-pass

GCS Secret: pgbackrest-gcs-key, key gcs-key.json

Recovery targets: day4-full-recovery and day4-pitr-recovery

Backup concepts

A logical backup contains SQL objects and data. `pg_dump` backs up one database; `pg_dumpall` also covers cluster-wide roles and tablespaces. A custom-format dump is restored with `pg_restore`. A plain SQL dump is restored with `psql`. Physical backups contain PostgreSQL data files and require compatible PostgreSQL binaries, extensions and WAL.

A full backup establishes a baseline. A differential backup contains changes since the latest full backup. An incremental backup contains changes since the preceding backup. `pgBackRest` resolves the dependency chain during restore; do not manually extract only the last incremental set. WAL archiving provides transaction history after the base backup. Replication and Patroni failover do not undo accidental `DELETE` or `DROP` operations.

RPO is acceptable data loss; actual recoverability is bounded by the last available WAL. RTO is acceptable recovery time and includes provisioning, download, replay, validation and application reconnection. A successful backup alone does not prove either target.

Lab 0 Workstation and cluster validation

Objective: establish one repeatable shell environment and confirm a healthy PostgreSQL 18 source before changing backup configuration.

0.1 Choose the correct terminal

Use the Ubuntu terminal that already has working access to Docker Kubernetes, or WSL with Docker Desktop integration and the correct kubeconfig. Cloud Shell can manage GCP but normally cannot reach your laptop's Docker Kubernetes cluster. Run kubectl, Docker and gcloud in the same configured workstation environment for this procedure.

```
bash
mkdir -p "$HOME/day4-complete"
cd "$HOME/day4-complete"
umask 077
mkdir -p backups/logical backups/physical evidence manifests
for tool in kubectl jq python3 openssl gcloud docker; do
  command -v "$tool" || echo "MISSING: $tool"
done
kubectl config current-context
kubectl cluster-info
docker version
gcloud version
```

Expected: the intended Kubernetes context, reachable API server and working tools. If a tool is missing, install it using its official workstation installer before continuing. Do not change the Kubernetes context merely to make a command succeed.

0.2 Save environment variables and inspect the container user

All Kubernetes operations below use kubectl explicitly. lab-env.sh contains only environment variables; it defines no command aliases or functions. The live Spilo commands show runuser -u postgres -- to execute as the Linux postgres user when the container starts commands as root. This prevents root-owned backup files and peer-authentication failures.

```
cat > lab-env.sh <<'EOF'
export NS=postgres-ha
export CLUSTER=acid-postgres-ha
export PRIMARY_SERVICE=acid-postgres-ha
export STANZA=postgres-ha
export PGBR_CONF=/etc/pgbackrest/pgbackrest.conf
export BUCKET=postgres-ha-backups-parameswari-2026
export REGION=asia-south1
EOF
source ./lab-env.sh
kubectl config set-context --current --namespace="$NS"
kubectl -n "$NS" get pods -o wide
kubectl -n "$NS" get svc,postgresql,pvc,statefulset
```

```
kubectl get storageclass
kubectl -n "$NS" exec acid-postgres-ha-0 -c postgres -- id
kubectl -n "$NS" exec acid-postgres-ha-0 -c postgres -- \
    id postgres
```

If the first id reports uid=0, use the live-cluster commands exactly as printed and verify runuser is available with the next command. If it already reports the postgres UID, omit only runuser -u postgres -- from the live-cluster commands. If the root image has gosu instead of runuser, replace that prefix with gosu postgres. Do not change the Kubernetes container user or permissions on live PGDATA merely to run a command.

```
kubectl -n "$NS" exec acid-postgres-ha-0 -c postgres -- \
    sh -c 'command -v runuser; command -v gosu'
```

The separately created recovery and disposable pods already run as the discovered postgres UID, so their kubectl commands do not use runuser. The -U postgres option in psql selects the database user; it does not select the Linux user.

0.2A Discover the current primary explicitly

Run this block now and whenever it appears before a backup or after a restart/switchover. It refreshes the pod list and requires exactly one writable member. It never assumes pod-0 is primary.

```
PODS=$(kubectl -n "$NS" get pods -l cluster-name="$CLUSTER" \
    -o jsonpath='{range .items[*]}{.metadata.name}{"\n"}{end}')
PRIMARY=""
PRIMARY_COUNT=0
for pod in $PODS; do
    ROLE=$(kubectl -n "$NS" exec "$pod" -c postgres -- \
        runuser -u postgres -- psql -X -U postgres -d postgres \
        -Atqc 'SELECT pg_is_in_recovery();') || exit 1
    if [ "$ROLE" = "f" ]; then
        PRIMARY="$pod"
        PRIMARY_COUNT=$((PRIMARY_COUNT + 1))
    fi
done
if [ "$PRIMARY_COUNT" -ne 1 ]; then
    echo "STOP: expected exactly one primary; found $PRIMARY_COUNT"
    exit 1
fi
export PRIMARY
printf 'Current primary: %s\n' "$PRIMARY"
```

Expected: exactly one primary and healthy replicas. If the label returns no pods, inspect kubectl get pods --show-labels and correct the cluster-name selector to match the actual live cluster before proceeding.

0.3 Validate roles and discover real paths

```

for pod in $PODS; do
    echo "=== $pod ==="
    kubectl -n "$NS" exec -i "$pod" -c postgres -- \
    runuser -u postgres -- id
    kubectl -n "$NS" exec -i "$pod" -c postgres -- \
    runuser -u postgres -- psql -X -U postgres -d postgres -Atqc \
    'SELECT pg_is_in_recovery(); SHOW server_version;'
done
REPLICA=$(printf '%s\n' "$PODS" \
    | grep -vx "$PRIMARY" | head -n 1)
export REPLICA
PGDATA_ACTUAL=$(kubectl -n "$NS" exec -i "$PRIMARY" -c postgres -- \
    runuser -u postgres -- psql -X -U postgres \
    -d postgres -Atqc 'SHOW data_directory;')
export PGDATA_ACTUAL
printf 'Replica: %s\nPGDATA: %s\n' "$REPLICA" "$PGDATA_ACTUAL"
kubectl -n "$NS" exec -i "$PRIMARY" -c postgres -- \
    runuser -u postgres -- psql -X -U postgres -d postgres <<'SQL'
\set ON_ERROR_STOP on
SHOW server_version_num;
SHOW config_file;
SHOW hba_file;
SHOW unix_socket_directories;
SELECT now(), pg_current_wal_lsn(), pg_is_in_recovery();
SELECT application_name, client_addr, state, sync_state,
    sent_lsn, write_lsn, flush_lsn, replay_lsn,
    pg_size_pretty(
        pg_wal_lsn_diff(pg_current_wal_lsn(), replay_lsn)
    ) AS replay_lag
FROM pg_stat_replication;
SELECT spcname, pg_tablespace_location(oid)
FROM pg_tablespace;
SELECT datname FROM pg_database ORDER BY datname;
SQL

```

Expected: PostgreSQL 18, primary returns f, replicas return t, and replication is streaming. PGDATA is often /home/postgres/pgdata/pgroot/data; the discovered value is authoritative. External tablespaces are recorded because physical restores must remap them onto recovery storage.

0.4 Save source configuration and initialize only training data

```
kubectl -n "$NS" get postgresql "$CLUSTER" -o yaml \
  > evidence/live-cr-before.yaml
kubectl -n "$NS" get statefulset "$CLUSTER" -o yaml \
  > evidence/live-sts-before.yaml
kubectl -n "$NS" exec -i "$PRIMARY" -c postgres -- \
  runuser -u postgres -- patronictl show-config \
  > evidence/patroni-before.yaml
kubectl -n "$NS" exec -i "$PRIMARY" -c postgres -- \
  runuser -u postgres -- psql -X -U postgres -d postgres \
  -Atqc "SELECT 1 FROM pg_database WHERE datname='banking';"
```

If banking does not exist, create it for this lab. If it already exists, skip the following creation command.

```
kubectl -n "$NS" exec -i "$PRIMARY" -c postgres -- \
  runuser -u postgres -- createdb -U postgres banking
```

Create a dedicated training schema and seed table. This avoids deleting or overwriting application tables named `dr_lab_events`.

```
kubectl -n "$NS" exec -i "$PRIMARY" -c postgres -- \
  runuser -u postgres -- psql -X -U postgres -d banking <<'SQL'
\set ON_ERROR_STOP on
CREATE SCHEMA IF NOT EXISTS day4_lab;
CREATE TABLE IF NOT EXISTS day4_lab.events (
  id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  event_time timestamptz NOT NULL DEFAULT clock_timestamp(),
  note text NOT NULL
);
INSERT INTO day4_lab.events(note) VALUES ('baseline before backups');
TABLE day4_lab.events;
SQL
```

Lab 0 checkpoint: save the detected source version, PGDATA, primary, replica, tablespaces and Kubernetes context. Reopen a terminal by returning to `day4-complete` and running source `./lab-env.sh` followed by the primary-discovery block in 0.2A. Password variables are intentionally not persisted.

Lab 1 Logical backup and restore

Objective: create portable database dumps and verify both archive contents and restored data.

1.1 Full cluster SQL dump and globals

Run on the workstation. Redirection writes locally; no terminal allocation is used for dump data.

```
PODS=$(kubectl -n "$NS" get pods -l cluster-name="$CLUSTER" \
-o jsonpath='{range .items[*]}{.metadata.name}{"\n"}{end}')
PRIMARY=""
PRIMARY_COUNT=0
for pod in $PODS; do
    ROLE=$(kubectl -n "$NS" exec "$pod" -c postgres -- \
    runuser -u postgres -- psql -X -U postgres -d postgres \
    -Atqc 'SELECT pg_is_in_recovery();') || exit 1
    if [ "$ROLE" = "f" ]; then
        PRIMARY="$pod"
        PRIMARY_COUNT=$((PRIMARY_COUNT + 1))
    fi
done
if [ "$PRIMARY_COUNT" -ne 1 ]; then
    echo "STOP: expected exactly one primary; found $PRIMARY_COUNT"
    exit 1
fi
export PRIMARY
printf 'Current primary: %s\n' "$PRIMARY"
STAMP=$(date -u +%Y%m%dT%H%M%S)
SQL_FILE="backups/logical/${CLUSTER}-${STAMP}.sql"
kubectl -n "$NS" exec -i "$PRIMARY" -c postgres -- \
    runuser -u postgres -- pg_dumpall -U postgres --clean --if-exists \
    > "$SQL_FILE"
kubectl -n "$NS" exec -i "$PRIMARY" -c postgres -- \
    runuser -u postgres -- pg_dumpall -U postgres --globals-only \
    > "backups/logical/globals-${STAMP}.sql"
test -s "$SQL_FILE"
ls -lh backups/logical
head -n 20 "$SQL_FILE"
```

The full SQL dump contains destructive DROP statements because --clean was requested in the source manuals. Keep it for a separate recovery target; do not replay it into the live cluster. Role password hashes in globals require confidential storage.

1.2 Custom format banking dump

```
BACKUP_FILE="backups/logical/banking-${STAMP}.dump"
kubectl -n "$NS" exec -i "$PRIMARY" -c postgres -- \
  runuser -u postgres -- pg_dump -U postgres -Fc -d banking \
  > "$BACKUP_FILE"
test -s "$BACKUP_FILE"
cat "$BACKUP_FILE" | kubectl -n "$NS" exec -i "$PRIMARY" -c postgres -- \
  runuser -u postgres -- \
  pg_restore --list > evidence/banking-dump-contents.txt
head -n 40 evidence/banking-dump-contents.txt
sha256sum "$BACKUP_FILE" > "${BACKUP_FILE}.sha256"
```

Expected: a nonempty custom archive with TABLE, TABLE DATA and SEQUENCE entries, including day4_lab.events. This uses the PostgreSQL client inside the pod, so no matching pg_restore installation is required on the workstation.

1.3 Restore into a new test database

The source manuals used banking_restore_test on the same live cluster. That verifies restore syntax but consumes live resources. The following uniquely named test database preserves that exercise without dropping an existing database. Appendix A also gives a separate-cluster option.

```
TEST_DB="banking_restore_$(date -u +%Y%m%d%H%M%S)"
kubectl -n "$NS" exec -i "$PRIMARY" -c postgres -- \
  runuser -u postgres -- createdb -U postgres "$TEST_DB"
cat "$BACKUP_FILE" | kubectl -n "$NS" exec -i "$PRIMARY" -c postgres -- \
  runuser -u postgres -- pg_restore \
  -U postgres -d "$TEST_DB" --exit-on-error \
  --no-owner --no-privileges
kubectl -n "$NS" exec -i "$PRIMARY" -c postgres -- \
  runuser -u postgres -- psql -X -U postgres -d "$TEST_DB" \
  -c '\dt day4_lab.*'
kubectl -n "$NS" exec -i "$PRIMARY" -c postgres -- \
  runuser -u postgres -- psql -X -U postgres -d "$TEST_DB" \
  -c 'TABLE day4_lab.events;'
printf '%s\n' "$TEST_DB" > evidence/logical-test-db.txt
```

Expected: baseline data appears in the new database. --no-owner and --no-privileges make this a data and schema drill; a production logical restore must restore required roles and grants deliberately. If source tablespaces are unavailable on an independent target, add --no-tablespaces. Never apply the full globals file without reviewing role and tablespace conflicts.

Lab 2 Physical backup using pg_basebackup

Objective: take a physical copy of the running primary using the replication protocol and copy the archive out of the pod.

2.1 Validate the Service and replication authentication

```
PODS=$(kubectl -n "$NS" get pods -l cluster-name="$CLUSTER" \
-o jsonpath='{range .items[*]}{.metadata.name}{"\n"}{end}')
PRIMARY=""
PRIMARY_COUNT=0
for pod in $PODS; do
    ROLE=$(kubectl -n "$NS" exec "$pod" -c postgres -- \
    runuser -u postgres -- psql -X -U postgres -d postgres \
    -Atqc 'SELECT pg_is_in_recovery();') || exit 1
    if [ "$ROLE" = "f" ]; then
        PRIMARY="$pod"
        PRIMARY_COUNT=$((PRIMARY_COUNT + 1))
    fi
done
if [ "$PRIMARY_COUNT" -ne 1 ]; then
    echo "STOP: expected exactly one primary; found $PRIMARY_COUNT"
    exit 1
fi
export PRIMARY
printf 'Current primary: %s\n' "$PRIMARY"
kubectl -n "$NS" get service "$PRIMARY_SERVICE"
kubectl -n "$NS" exec -i "$REPLICA" -c postgres -- \
    runuser -u postgres -- getent hosts "$PRIMARY_SERVICE"
kubectl -n "$NS" exec -i "$PRIMARY" -c postgres -- \
    runuser -u postgres -- psql -X -U postgres -d postgres <<'SQL'
\set ON_ERROR_STOP on
SELECT rolname, rolreplication, rolcanlogin
FROM pg_roles WHERE rolreplication;
SELECT line_number, type, database, user_name,
       address, auth_method, error
FROM pg_hba_file_rules
WHERE 'replication' = ANY(database);
SQL
kubectl -n "$NS" get secrets | grep -Ei 'standby|replic'
```

The host is the Service acid-postgres-ha, not the namespace postgres-ha. localhost inside the replica means the replica itself. Use the Secret belonging to this cluster; change REPL_SECRET if the displayed name differs.

```
REPL_SECRET=standby.acid-postgres-ha.credentials.postgresql.acid.zalan.do
REPL_USER=$(kubectl -n "$NS" get secret "$REPL_SECRET" \
-o jsonpath='{.data.username}' | base64 -d)
REPL_PASS=$(kubectl -n "$NS" get secret "$REPL_SECRET" \
-o jsonpath='{.data.password}' | base64 -d)
```

```
test -n "$REPL_USER" && test -n "$REPL_PASS"
printf 'Replication user: %s\n' "$REPL_USER"
```

Do not print REPL_PASS or enable shell tracing. The lab passes it through kubectl exec environment arguments; use a managed secret/passfile workflow for production automation. sslmode=require encrypts this lab connection; it does not verify server identity like verify-full with a trusted CA.

2.2 Optional ordinary connection check

```
kubectl -n "$NS" exec -i "$REPLICA" -c postgres -- \
  runuser -u postgres -- env PGPASSWORD="$REPL_PASS" \
  PGSSLMODE=require psql -X \
  -h "$PRIMARY_SERVICE" -p 5432 -U "$REPL_USER" \
  -d postgres -c 'SELECT pg_is_in_recovery();'
```

If allowed, this returns f. A replication-only HBA rule may deliberately reject ordinary SQL connections; that alone does not invalidate pg_basebackup. The backup in 2.3 is the definitive replication connection test. Do not add broad trust rules to bypass a rejected connection.

2.3 Create a fresh backup directory and run the backup

```
BASE_DIR="/tmp/day4-basebackup-$(date -u +%Y%m%d%H%M%S)"
kubectl -n "$NS" exec -i "$REPLICA" -c postgres -- \
  runuser -u postgres -- mkdir -m 700 "$BASE_DIR"
kubectl -n "$NS" exec -i "$REPLICA" -c postgres -- \
  runuser -u postgres -- df -h /tmp
kubectl -n "$NS" exec -i "$REPLICA" -c postgres -- \
  runuser -u postgres -- env PGPASSWORD="$REPL_PASS" \
  PGSSLMODE=require pg_basebackup \
  -h "$PRIMARY_SERVICE" -p 5432 -U "$REPL_USER" \
  -D "$BASE_DIR" -Ft -z -Xs -P --checkpoint=fast
kubectl -n "$NS" exec -i "$REPLICA" -c postgres -- \
  runuser -u postgres -- ls -lh "$BASE_DIR"
```

-Ft creates tar archives; -z compresses them; -Xs streams required WAL concurrently; -P shows progress. The fast checkpoint can increase source I/O. Ensure /tmp has enough space first; use dedicated backup storage for large clusters. Tablespaces can produce additional OID-named archives.

2.4 Export and check the archive files

```

PHYSICAL_DIR="backups/physical/basebackup-$(date -u +%Y%m%d%H%M%S) "
mkdir -p "$PHYSICAL_DIR"
kubectl -n "$NS" cp -c postgres \
    "$REPLICA:$BASE_DIR/." "$PHYSICAL_DIR/"
find "$PHYSICAL_DIR" -maxdepth 1 -type f -ls
for archive in "$PHYSICAL_DIR"/*.tar.gz; do
    gzip -t "$archive" || break
done
sha256sum "$PHYSICAL_DIR"/* > evidence/basebackup-sha256.txt
unset REPL_PASS

```

Expected: base.tar.gz, streamed WAL archive and any tablespace archives. A gzip test proves transport integrity, not recoverability. Lab 8 performs an actual physical restore using pgBackRest; this pg_basebackup copy is a separate artifact and cannot be passed to pgBackRest restore. Preserve its backup manifest if emitted. pg_verifybackup requires a correctly extracted backup and mapped tablespaces, not the compressed tar directory as-is.

Lab 3 Inspect existing WAL archiving

Objective: record the current archive destination before replacing anything and distinguish working archives from a successful no-op command.

3.1 Inspect PostgreSQL settings and native tools

```
PODS=$(kubectl -n "$NS" get pods -l cluster-name="$CLUSTER" \
-o jsonpath='{range .items[*]}{.metadata.name}{"\n"}{end}')
PRIMARY=""
PRIMARY_COUNT=0
for pod in $PODS; do
    ROLE=$(kubectl -n "$NS" exec "$pod" -c postgres -- \
    runuser -u postgres -- psql -X -U postgres -d postgres \
    -Atqc 'SELECT pg_is_in_recovery();') || exit 1
    if [ "$ROLE" = "f" ]; then
        PRIMARY="$pod"
        PRIMARY_COUNT=$((PRIMARY_COUNT + 1))
    fi
done
if [ "$PRIMARY_COUNT" -ne 1 ]; then
    echo "STOP: expected exactly one primary; found $PRIMARY_COUNT"
    exit 1
fi
export PRIMARY
printf 'Current primary: %s\n' "$PRIMARY"
kubectl -n "$NS" exec -i "$PRIMARY" -c postgres -- \
    runuser -u postgres -- psql -X -U postgres -d postgres <<'SQL'
\set ON_ERROR_STOP on
SHOW archive_mode;
SHOW archive_command;
SHOW archive_library;
SELECT * FROM pg_stat_archiver;
SQL
kubectl -n "$NS" exec -i "$PRIMARY" -c postgres -- \
    runuser -u postgres -- sh -c '
    command -v wal-g || true
    command -v wal-e || true
    printenv | cut -d= -f1 | grep -E "WALG|WALE|WAL_" || true
'

kubectl -n "$NS" exec -i "$PRIMARY" -c postgres -- \
    runuser -u postgres -- psql -X -U postgres -d postgres -Atqc \
    'SHOW archive_command;' > evidence/archive-command-before.txt
```

Environment variable names are shown without secret values. If WAL-G/WAL-E is already backing up important data, the pgBackRest archive switch in Lab 4 is a deliberate migration step for this training cluster. Keep the old repository intact and do not assume its old backups are recoverable through pgBackRest.

3.2 Force WAL activity and observe the archiver

```
BEFORE=$(kubectl -n "$NS" exec -i "$PRIMARY" -c postgres -- \
  runuser -u postgres -- psql -X -U postgres -d postgres \
  -Atqc 'SELECT archived_count FROM pg_stat_archiver;')
kubectl -n "$NS" exec -i "$PRIMARY" -c postgres -- \
  runuser -u postgres -- psql -X -U postgres -d banking <<'SQL'
\set ON_ERROR_STOP on
INSERT INTO day4_lab.events(note) VALUES ('Lab 3 WAL probe');
SELECT pg_switch_wal();
SQL
sleep 10
kubectl -n "$NS" exec -i "$PRIMARY" -c postgres -- \
  runuser -u postgres -- psql -X -U postgres -d postgres -x -c \
  'SELECT archived_count, failed_count, last_archived_wal,
    last_archived_time, last_failed_wal, last_failed_time
    FROM pg_stat_archiver;'
printf 'Archived before test: %s\n' "$BEFORE"
```

If `archive_mode` is off, or `archive_command` is `/bin/true`, this does not prove usable archived WAL. `/bin/true` can increase success counters while storing nothing. Lab 4 establishes repository-backed checks. A quiet cluster might not switch a segment without WAL activity, which is why this step inserts a marker first.

3.3 Native operator backup clarification

The original Kubernetes manual used the annotation `force-backup.acid.zalan.do`. Do not treat that annotation as a universal Zalando backup trigger; arbitrary annotations may be accepted without any backup running. Native WAL-G backup requires the installed Spilo backup configuration and its documented backup job. If you keep the native path, inspect the deployed image and schedules, then verify a new backup with that tool's `backup-list` and a restore test. This manual's executable backup path is `pgBackRest` in Labs 4 through 9.

Lab 4 Configure pgBackRest completely

Objective: make the binary, image, credentials, repository, mounts and archive command consistent on every member before taking a backup.

4.1 Capture image and inspect installed pgBackRest

```
PODS=$(kubectl -n "$NS" get pods -l cluster-name="$CLUSTER" \
-o jsonpath='{range .items[*]}{.metadata.name}{"\n"}{end}')
PRIMARY=""
PRIMARY_COUNT=0
for pod in $PODS; do
    ROLE=$(kubectl -n "$NS" exec "$pod" -c postgres -- \
    runuser -u postgres -- psql -X -U postgres -d postgres \
    -Atqc 'SELECT pg_is_in_recovery();') || exit 1
    if [ "$ROLE" = "f" ]; then
        PRIMARY="$pod"
        PRIMARY_COUNT=$((PRIMARY_COUNT + 1))
    fi
done
if [ "$PRIMARY_COUNT" -ne 1 ]; then
    echo "STOP: expected exactly one primary; found $PRIMARY_COUNT"
    exit 1
fi
export PRIMARY
printf 'Current primary: %s\n' "$PRIMARY"
CURRENT_IMAGE=$(kubectl -n "$NS" get pod "$PRIMARY" \
-o jsonpath='{.spec.containers[?(@.name=="postgres")].image}')
export CURRENT_IMAGE
printf '%s\n' "$CURRENT_IMAGE" > evidence/source-image.txt
for pod in $PODS; do
    echo "=== $pod ==="
    kubectl -n "$NS" exec -i "$pod" -c postgres -- \
    runuser -u postgres -- pgbackrest version
    kubectl -n "$NS" exec -i "$pod" -c postgres -- \
    runuser -u postgres -- sh -c 'cat /etc/os-release'
done
```

The dry-run manuals reported pgBackRest 2.59.1; verify the installed version rather than assuming it. All live and recovery images need a PostgreSQL 18-compatible build. If every member already has a suitable version in its immutable image, skip the build and live image patch in 4.5, but still complete GCS, mounts and recovery steps. Recovery uses the same live image to retain extension libraries.

4.2 Authenticate to GCP and choose the actual project

```
gcloud auth login
gcloud projects list
read -r -p 'Enter the existing GCP project ID: ' PROJECT_ID
export PROJECT_ID
test -n "$PROJECT_ID"
gcloud config set project "$PROJECT_ID"
gcloud config get-value project
gcloud services enable storage.googleapis.com \
  artifactregistry.googleapis.com iam.googleapis.com \
  --project="$PROJECT_ID"
```

Use the project ID displayed by your account, not a guessed project name. Earlier sessions used more than one spelling of the project. The required permissions include enabling services, creating a bucket/repository/service account and granting the specific IAM bindings below. If permission is denied, have the project administrator grant the necessary lab access.

4.3 Create or verify the GCS backup bucket

```
gcloud storage buckets list --project="$PROJECT_ID"
gcloud storage buckets describe "gs://$BUCKET"
```

If this existing bucket is owned by your intended project, reuse it. If it does not exist, run the next block. A 403 is an access problem, not evidence that a new bucket should be created. Bucket names are globally unique; if another owner has the name, choose your own name, update BUCKET in lab-env.sh and source it again.

```
gcloud storage buckets create "gs://$BUCKET" \
  --project="$PROJECT_ID" --location="$REGION" \
  --uniform-bucket-level-access
gcloud storage buckets describe "gs://$BUCKET"
```

GCS holds backup data and archived WAL. Artifact Registry, created later only when needed, holds container images. They are different resources. Do not add a bucket lifecycle rule that deletes objects sooner than pgBackRest retention permits.

4.4 Create or reuse the GCS identity and encryption Secret

```
SA_NAME=pgbackrest-gcs
SA_EMAIL="${SA_NAME}@${PROJECT_ID}.iam.gserviceaccount.com"
export SA_EMAIL
gcloud iam service-accounts list --project="$PROJECT_ID"
gcloud iam service-accounts describe "$SA_EMAIL" \
  --project="$PROJECT_ID"
```

Only if that service account is absent, create it:

```
gcloud iam service-accounts create "$SA_NAME" \
  --project="$PROJECT_ID" \
  --display-name='pgBackRest GCS lab backup'
```

Grant access on the backup bucket, then inspect existing Kubernetes Secrets.

```
gcloud storage buckets add-iam-policy-binding "gs://$BUCKET" \
  --member="serviceAccount:$SA_EMAIL" \
  --role=roles/storage.objectAdmin
kubectl -n "$NS" get secret pgbackrest-gcs-key
kubectl -n "$NS" get secret pgbackrest-repo-secret
```

If pgbackrest-gcs-key already exists with the correct identity, retain it. If it is missing, create the JSON key and Secret below. Docker Kubernetes cannot use Google VM metadata credentials automatically. This lab uses key-type=service. If organization policy blocks key creation, stop this branch and ask the organization's administrator for an approved identity method or approved lab-project exception. Do not disable organization security policy as a routine troubleshooting step.

```
gcloud iam service-accounts keys create pgbackrest-gcs-key.json \
  --iam-account="$SA_EMAIL" --project="$PROJECT_ID"
chmod 600 pgbackrest-gcs-key.json
kubectl -n "$NS" create secret generic pgbackrest-gcs-key \
  --from-file=gcs-key.json=./pgbackrest-gcs-key.json
```

If pgbackrest-repo-secret is absent and this is a NEW repository, create it once:

```
PGBR_CIPHER_PASS=$(openssl rand -base64 48)
kubectl -n "$NS" create secret generic pgbackrest-repo-secret \
  --from-literal=repo-cipher-pass="$PGBR_CIPHER_PASS"
unset PGBR_CIPHER_PASS
```

If encrypted backups already exist and the cipher Secret is missing, recover the original passphrase from your secure record. A new passphrase will not decrypt the old repository. Store a protected recovery copy of both credentials outside the Kubernetes cluster; Kubernetes Secrets are not themselves an off-cluster backup.

```
kubectl -n "$NS" get secret pgbackrest-repo-secret -o json \
  | jq -e '.data["repo-cipher-pass"] | length > 0'
kubectl -n "$NS" get secret pgbackrest-gcs-key -o json \
  | jq -e '.data["gcs-key.json"] | length > 0'
```

Expected: both checks print true. Do not display the decoded cipher or full JSON key.

4.5 Optional immutable image build and registry setup

Run this entire section only when the live image lacks a suitable pgBackRest binary. It is retained explicitly because earlier versions omitted image creation. The Dockerfile inherits the exact deployed Spilo image and restores its original image USER; changing Spilo's entrypoint user unconditionally can break its initialization.

```
AR_REPO=postgres-lab-images
REGISTRY="${REGION}-docker.pkg.dev"
export AR_REPO REGISTRY
gcloud artifacts repositories list --location="$REGION" \
```

```
--project="$PROJECT_ID"
```

If postgres-lab-images is absent, run the repository create command below; if it exists, skip that one command and continue with authentication and the image build:

```
gcloud artifacts repositories create "$AR_REPO" \
  --repository-format=docker --location="$REGION" \
  --project="$PROJECT_ID" \
  --description='PostgreSQL training images'
gcloud auth configure-docker "$REGISTRY"
docker pull "$CURRENT_IMAGE"
BASE_USER=$(docker image inspect "$CURRENT_IMAGE" \
  --format '{{.Config.User}}')
BASE_USER=${BASE_USER:-root}
cat > Dockerfile.spilo-pgbackrest <<'EOF'
ARG BASE_IMAGE
FROM ${BASE_IMAGE}
ARG BASE_USER=root
USER root
RUN apt-get update \
  && apt-get install -y --no-install-recommends pgbackrest \
  && pgbackrest version \
  && rm -rf /var/lib/apt/lists/*
USER ${BASE_USER}
EOF
CUSTOM_IMAGE="${REGISTRY}/${PROJECT_ID}/${AR_REPO}/spilo-pgbackrest:day4-pg18"
export CUSTOM_IMAGE
docker build --build-arg BASE_IMAGE="$CURRENT_IMAGE" \
  --build-arg BASE_USER="$BASE_USER" \
  -f Dockerfile.spilo-pgbackrest -t "$CUSTOM_IMAGE" .
docker run --rm --entrypoint pgbackrest "$CUSTOM_IMAGE" version
```

This build path assumes the Debian/Ubuntu Spilo base observed in the supplied lab. Check the package's version and PostgreSQL 18 support before deployment. If apt supplies an incompatible package, do not deploy it; use a compatible package from the base distribution's approved PostgreSQL repository or a reviewed pgBackRest source build. Do not silently install a package at every pod startup.

Publish only after the binary version is suitable:

```
docker push "$CUSTOM_IMAGE"
gcloud artifacts docker images list \
  "${REGISTRY}/${PROJECT_ID}/${AR_REPO}" --include-tags
```

For local Kubernetes to pull the private image, use an image pull Secret. The next block uses a short-lived access token from your authenticated workstation identity, which needs Artifact Registry read permission. Tokens expire, so refresh this Secret before later pod creation. It is separate from the GCS JSON credential.

```
kubectl -n "$NS" create secret docker-registry day4-registry \
  --docker-server="$REGISTRY" \
```

```

--docker-username=oauth2accesstoken \
--docker-password="$(gcloud auth print-access-token)" \
--dry-run=client -o yaml | kubectl apply -f -
POD_SA=$(kubectl -n "$NS" get pod "$PRIMARY" \
-o jsonpath='{.spec.serviceAccountName}')
POD_SA=${POD_SA:-default}
kubectl -n "$NS" get serviceaccount "$POD_SA" -o json \
> evidence/pod-sa.json
python3 - <<'PY'
import json
s=json.load(open('evidence/pod-sa.json'))
p=s.get('imagePullSecrets', [])
if not any(x['name']=='day4-registry' for x in p):
    p.append({'name':'day4-registry'})
json.dump({'imagePullSecrets':p},
          open('manifests/pull-secret-patch.json','w'), indent=2)
PY
kubectl -n "$NS" patch serviceaccount "$POD_SA" \
--type=merge --patch-file=manifests/pull-secret-patch.json
jq -n --arg image "$CUSTOM_IMAGE" \
'{spec:{dockerImage:$image}}' > manifests/image-patch.json
kubectl -n "$NS" patch postgresql "$CLUSTER" --type=merge \
--patch-file=manifests/image-patch.json --dry-run=server
kubectl -n "$NS" patch postgresql "$CLUSTER" --type=merge \
--patch-file=manifests/image-patch.json
kubectl -n "$NS" get pods -w

```

Stop the watch with Ctrl+C after operator reconciliation and all members are Ready. Verify actual pod images and version on every member. A rolling image change can move the primary. Do not manually replace the operator-owned StatefulSet.

```

PODS=$(kubectl -n "$NS" get pods -l cluster-name="$CLUSTER" \
-o jsonpath='{range .items[*]}{.metadata.name}{"\n"}{end}')
PRIMARY=""
PRIMARY_COUNT=0
for pod in $PODS; do
    ROLE=$(kubectl -n "$NS" exec "$pod" -c postgres -- \
    runuser -u postgres -- psql -X -U postgres -d postgres \
    -Atqc 'SELECT pg_is_in_recovery();') || exit 1
    if [ "$ROLE" = "f" ]; then
        PRIMARY="$pod"
        PRIMARY_COUNT=$((PRIMARY_COUNT + 1))
    fi
done
if [ "$PRIMARY_COUNT" -ne 1 ]; then
    echo "STOP: expected exactly one primary; found $PRIMARY_COUNT"
    exit 1
fi
export PRIMARY
printf 'Current primary: %s\n' "$PRIMARY"
for pod in $PODS; do
    kubectl -n "$NS" get pod "$pod" \

```

```

    -o jsonpath='{.spec.containers[?(@.name=="postgres")].image}'
echo
kubectl -n "$NS" exec -i "$pod" -c postgres -- \
runuser -u postgres -- pgbackrest version
done

```

If the image was already suitable and you skipped 4.5, simply continue at 4.6. Later recovery discovery reads the image actually running, so no unset CUSTOM_IMAGE is required.

4.6 Generate one consistent pgBackRest configuration

For an existing repository, confirm its bucket, prefix, cipher type and passphrase match this configuration before applying it. Do not reinitialize an existing stanza with a different PostgreSQL system identifier.

```

PODS=$(kubectl -n "$NS" get pods -l cluster-name="$CLUSTER" \
-o jsonpath='{range .items[*]}{.metadata.name}{"\n"}{end}')
PRIMARY=""
PRIMARY_COUNT=0
for pod in $PODS; do
    ROLE=$(kubectl -n "$NS" exec "$pod" -c postgres -- \
        runuser -u postgres -- psql -X -U postgres -d postgres \
        -Atqc 'SELECT pg_is_in_recovery();') || exit 1
    if [ "$ROLE" = "f" ]; then
        PRIMARY="$pod"
        PRIMARY_COUNT=$((PRIMARY_COUNT + 1))
    fi
done
if [ "$PRIMARY_COUNT" -ne 1 ]; then
    echo "STOP: expected exactly one primary; found $PRIMARY_COUNT"
    exit 1
fi
export PRIMARY
printf 'Current primary: %s\n' "$PRIMARY"
PGDATA_ACTUAL=$(kubectl -n "$NS" exec -i "$PRIMARY" -c postgres -- \
    runuser -u postgres -- psql -X -U postgres \
    -d postgres -Atqc 'SHOW data_directory;')
export PGDATA_ACTUAL
cat > pgbackrest.conf <<EOF
[postgres-ha]
pg1-path=$PGDATA_ACTUAL
pg1-port=5432
pg1-user=postgres

[global]
repl-type=gcs
repl-gcs-bucket=$BUCKET
repl-gcs-key-type=service
repl-gcs-key=/etc/pgbackrest-gcs/gcs-key.json
repl-path=/pgbackrest/postgres-ha
repl-retention-full=2

```

```

repol-retention-diff=4
repol-cipher-type=aes-256-cbc
compress-type=zst
compress-level=3
process-max=2
start-fast=y
archive-timeout=120
log-level-console=info
log-level-file=off
EOF
kubectl -n "$NS" create configmap pgbackrest-config \
  --from-file=pgbackrest.conf=./pgbackrest.conf \
  --dry-run=client -o yaml | kubectl apply -f -
cat pgbackrest.conf

```

log-level-file=off avoids writing to a root-owned default log directory; command logs can be captured on the workstation. archive-timeout is pgBackRest's wait for required WAL, not PostgreSQL's archive_timeout setting. Raising it does not repair an incorrect archive destination.

4.7 Merge mounts without dropping other additionalVolumes

The script below preserves unrelated mounts and replaces only the three named pgBackRest entries. JSON output avoids invalid YAML indentation and multiline JSON strings.

```

kubectl -n "$NS" get postgresql "$CLUSTER" -o json \
  > evidence/live-cr-current.json
python3 - <<'PY'
import json
c=json.load(open('evidence/live-cr-current.json'))
new=[
  {'name': 'pgbackrest-config',
   'mountPath': '/etc/pgbackrest',
   'targetContainers': ['postgres'],
   'volumeSource': {'configMap': {'name': 'pgbackrest-config'}}},
  {'name': 'pgbackrest-secret',
   'mountPath': '/etc/pgbackrest-secret',
   'targetContainers': ['postgres'],
   'volumeSource': {'secret': {
     'secretName': 'pgbackrest-repo-secret'}}},
  {'name': 'pgbackrest-gcs-key',
   'mountPath': '/etc/pgbackrest-gcs',
   'targetContainers': ['postgres'],
   'volumeSource': {'secret': {'secretName': 'pgbackrest-gcs-key'}}}
]
names={v['name'] for v in new}
vols=[v for v in c['spec'].get('additionalVolumes', [])
      if v['name'] not in names]
vols.extend(new)
json.dump({'spec': {'additionalVolumes': vols}},
          open('manifests/volumes-patch.json', 'w'), indent=2)

```

```

PY
python3 -m json.tool manifests/volumes-patch.json
kubectl -n "$NS" patch postgresql "$CLUSTER" --type=merge \
  --patch-file=manifests/volumes-patch.json \
  --dry-run=server -o yaml > evidence/volumes-preview.yaml
kubectl -n "$NS" patch postgresql "$CLUSTER" --type=merge \
  --patch-file=manifests/volumes-patch.json
kubectl -n "$NS" get pods -w

```

Wait for the operator to recreate/reconcile the pod templates. Ctrl+C ends the watch. A rollout status alone can return before the operator has processed the CR; the file checks below are the final gate.

```

for pod in $PODS; do
  echo "=== $pod ==="
  kubectl -n "$NS" exec -i "$pod" -c postgres -- \
  runuser -u postgres -- sh -ec '
    test -s /etc/pgbackrest/pgbackrest.conf
    test -r /etc/pgbackrest-secret/repo-cipher-pass
    test -r /etc/pgbackrest-gcs/gcs-key.json
    echo "All three files readable as postgres"
  '
done
PODS=$(kubectl -n "$NS" get pods -l cluster-name="$CLUSTER" \
  -o jsonpath='{range .items[*]}{.metadata.name}{"\n"}{end}')
PRIMARY=""
PRIMARY_COUNT=0
for pod in $PODS; do
  ROLE=$(kubectl -n "$NS" exec "$pod" -c postgres -- \
    runuser -u postgres -- psql -X -U postgres -d postgres \
    -Atqc 'SELECT pg_is_in_recovery();') || exit 1
  if [ "$ROLE" = "f" ]; then
    PRIMARY="$pod"
    PRIMARY_COUNT=$((PRIMARY_COUNT + 1))
  fi
done
if [ "$PRIMARY_COUNT" -ne 1 ]; then
  echo "STOP: expected exactly one primary; found $PRIMARY_COUNT"
  exit 1
fi
export PRIMARY
printf 'Current primary: %s\n' "$PRIMARY"

```

All three checks must pass on every member. A ConfigMap update can take time to reach an existing projected volume; a subPath mount would require pod replacement. This procedure mounts whole directories.

4.8 Create the stanza on the current primary

```
PODS=$(kubectl -n "$NS" get pods -l cluster-name="$CLUSTER" \
-o jsonpath='{range .items[*]}{.metadata.name}{"\n"}{end}')
PRIMARY=""
PRIMARY_COUNT=0
for pod in $PODS; do
    ROLE=$(kubectl -n "$NS" exec "$pod" -c postgres -- \
    runuser -u postgres -- psql -X -U postgres -d postgres \
    -Atqc 'SELECT pg_is_in_recovery();') || exit 1
    if [ "$ROLE" = "f" ]; then
        PRIMARY="$pod"
        PRIMARY_COUNT=$((PRIMARY_COUNT + 1))
    fi
done
if [ "$PRIMARY_COUNT" -ne 1 ]; then
    echo "STOP: expected exactly one primary; found $PRIMARY_COUNT"
    exit 1
fi
export PRIMARY
printf 'Current primary: %s\n' "$PRIMARY"
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- \
runuser -u postgres -- sh -ec '
    export PGBACKREST_REPO1_CIPHER_PASS="$(
        cat /etc/pgbackrest-secret/repo-cipher-pass
    )"
    exec pgbackrest \
        --config=/etc/pgbackrest/pgbackrest.conf \
        --stanza=postgres-ha stanza-create
'
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- \
runuser -u postgres -- sh -ec '
    export PGBACKREST_REPO1_CIPHER_PASS="$(
        cat /etc/pgbackrest-secret/repo-cipher-pass
    )"
    exec pgbackrest \
        --config=/etc/pgbackrest/pgbackrest.conf \
        --stanza=postgres-ha info
'
```

Expected: stanza-create succeeds. info can report no backups yet for a new repository. This step precedes changing archive_command; pgBackRest check comes after archiving is configured. If the repository reports a mismatched database system identifier, do not delete metadata: inspect the existing repository and select the intended cluster/repository.

4.9 Set archive_command using a validated patch

```
python3 - <<'PY'
import json
cmd=('PGBACKREST_REPO1_CIPHER_PASS="$(cat '
    '/etc/pgbackrest-secret/repo-cipher-pass)" '
    'pgbackrest --config=/etc/pgbackrest/pgbackrest.conf '
    '--stanza=postgres-ha archive-push "%p"')
patch={'spec':{'postgresql':{'parameters':{'
    'archive_mode':'on', 'archive_library':'',
    'archive_command':cmd}}}}
json.dump(patch, open('manifests/archive-patch.json','w'), indent=2)
PY
python3 -m json.tool manifests/archive-patch.json
kubectl -n "$NS" patch postgresql "$CLUSTER" --type=merge \
    --patch-file=manifests/archive-patch.json --dry-run=server
kubectl -n "$NS" patch postgresql "$CLUSTER" --type=merge \
    --patch-file=manifests/archive-patch.json
kubectl -n "$NS" get pods -w
```

After reconciliation, Ctrl+C and check the actual PostgreSQL settings. archive_mode requires restart if its prior value was off. Configuration reload alone does not enable it.

```
PODS=$(kubectl -n "$NS" get pods -l cluster-name="$CLUSTER" \
    -o jsonpath='{range .items[*]}{.metadata.name}{ "\n"}{end}')
PRIMARY=""
PRIMARY_COUNT=0
for pod in $PODS; do
    ROLE=$(kubectl -n "$NS" exec "$pod" -c postgres -- \
        runuser -u postgres -- psql -X -U postgres -d postgres \
        -Atqc 'SELECT pg_is_in_recovery();') || exit 1
    if [ "$ROLE" = "f" ]; then
        PRIMARY="$pod"
        PRIMARY_COUNT=$((PRIMARY_COUNT + 1))
    fi
done
if [ "$PRIMARY_COUNT" -ne 1 ]; then
    echo "STOP: expected exactly one primary; found $PRIMARY_COUNT"
    exit 1
fi
export PRIMARY
printf 'Current primary: %s\n' "$PRIMARY"
kubectl -n "$NS" exec -i "$PRIMARY" -c postgres -- \
    runuser -u postgres -- patronictl list
kubectl -n "$NS" exec -i "$PRIMARY" -c postgres -- \
    runuser -u postgres -- patronictl show-config
for pod in $PODS; do
    echo "=== $pod ==="
    kubectl -n "$NS" exec -i "$pod" -c postgres -- \
        runuser -u postgres -- psql -X -U postgres -d postgres -c \
        "SELECT name, setting, pending_restart
        FROM pg_settings"
```

```
WHERE name IN ('archive_mode','archive_command',
               'archive_library');"
done
```

If Patroni reports pending restart after the operator has reconciled, perform a controlled restart in the lab maintenance window, one replica at a time, wait for streaming, then switchover to a restarted replica and restart the former primary. Use `patronictl restart` interactively, selecting one member; do not force-restart all members together. If `show-config` still disagrees with the CR, use the troubleshooting steps before any backup.

```
kubectl -n "$NS" exec -it "$PRIMARY" -c postgres -- \
  patronictl restart "$CLUSTER"
```

Expected on all members: `archive_mode=on`, empty `archive_library` and the matching `/etc/pgbackrest` configuration plus stanza `postgres-ha`. No `/bin/true`, no old `/home/postgres/pgdata/pgbackrest.conf`, and no alternate stanza `acid-postgres-ha`.

4.10 Prove repository WAL archiving

```
PODS=$(kubectl -n "$NS" get pods -l cluster-name="$CLUSTER" \
  -o jsonpath='{range .items[*]}{.metadata.name}{"\n"}{end}')
PRIMARY=""
PRIMARY_COUNT=0
for pod in $PODS; do
  ROLE=$(kubectl -n "$NS" exec "$pod" -c postgres -- \
    runuser -u postgres -- psql -X -U postgres -d postgres \
    -Atqc 'SELECT pg_is_in_recovery();') || exit 1
  if [ "$ROLE" = "f" ]; then
    PRIMARY="$pod"
    PRIMARY_COUNT=$((PRIMARY_COUNT + 1))
  fi
done
if [ "$PRIMARY_COUNT" -ne 1 ]; then
  echo "STOP: expected exactly one primary; found $PRIMARY_COUNT"
  exit 1
fi
export PRIMARY
printf 'Current primary: %s\n' "$PRIMARY"
kubectl -n "$NS" exec -i "$PRIMARY" -c postgres -- \
  runuser -u postgres -- psql -X -U postgres -d banking <<'SQL'
\set ON_ERROR_STOP on
INSERT INTO day4_lab.events(note) VALUES ('pgBackRest archive probe');
SELECT pg_switch_wal();
SQL
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- \
  runuser -u postgres -- sh -ec '
  export PGBACKREST_REPO1_CIPHER_PASS="$(
    cat /etc/pgbackrest-secret/repo-cipher-pass
  )"
  exec pgbackrest \
    --config=/etc/pgbackrest/pgbackrest.conf \
```

```

    --stanza=postgres-ha check
,
kubectl -n "$NS" exec -i "$PRIMARY" -c postgres -- \
  runuser -u postgres -- psql -X -U postgres -d postgres -x -c \
  'SELECT archived_count, failed_count, last_archived_wal,
        last_archived_time, last_failed_wal, last_failed_time
    FROM pg_stat_archiver;'
gcloud storage ls \
  "gs://$BUCKET/pgbackrest/postgres-ha/archive/"

```

Expected: check succeeds, new WAL appears under archive/postgres-ha and archiver counters progress without new failures. Historical failed_count does not need to become zero. Do not start Lab 5 until these checks pass. Inspect the bucket here after stanza/archive setup, and again after each backup; an empty backup directory before the first backup is normal.

Lab 5 Full backup

Objective: establish a verified physical baseline with known data.

5.1 Write a baseline row before the backup

```
PODS=$(kubectl -n "$NS" get pods -l cluster-name="$CLUSTER" \
-o jsonpath='{range .items[*]}{.metadata.name}{"\n"}{end}')
PRIMARY=""
PRIMARY_COUNT=0
for pod in $PODS; do
    ROLE=$(kubectl -n "$NS" exec "$pod" -c postgres -- \
    runuser -u postgres -- psql -X -U postgres -d postgres \
    -Atqc 'SELECT pg_is_in_recovery();') || exit 1
    if [ "$ROLE" = "f" ]; then
        PRIMARY="$pod"
        PRIMARY_COUNT=$((PRIMARY_COUNT + 1))
    fi
done
if [ "$PRIMARY_COUNT" -ne 1 ]; then
    echo "STOP: expected exactly one primary; found $PRIMARY_COUNT"
    exit 1
fi
export PRIMARY
printf 'Current primary: %s\n' "$PRIMARY"
kubectl -n "$NS" exec -i "$PRIMARY" -c postgres -- \
    runuser -u postgres -- psql -X -U postgres -d banking \
    -v ON_ERROR_STOP=1 -c \
    "INSERT INTO day4_lab.events(note) VALUES ('Lab 5 full baseline');"
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- \
    runuser -u postgres -- sh -ec '
        export PGBACKREST_REPO1_CIPHER_PASS="$(
            cat /etc/pgbackrest-secret/repo-cipher-pass
        )"
        exec pgbackrest \
            --config=/etc/pgbackrest/pgbackrest.conf \
            --stanza=postgres-ha check
    '
,
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- \
    runuser -u postgres -- sh -ec '
        export PGBACKREST_REPO1_CIPHER_PASS="$(
            cat /etc/pgbackrest-secret/repo-cipher-pass
        )"
        exec pgbackrest \
            --config=/etc/pgbackrest/pgbackrest.conf \
            --stanza=postgres-ha --type=full backup
    '
,
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- \
    runuser -u postgres -- sh -ec '
        export PGBACKREST_REPO1_CIPHER_PASS="$(
            cat /etc/pgbackrest-secret/repo-cipher-pass
```

```

)"
exec pgbackrest \
  --config=/etc/pgbackrest/pgbackrest.conf \
  --stanza=postgres-ha info
,
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- \
runuser -u postgres -- sh -ec '
  export PGBACKREST_REP01_CIPHER_PASS="$(
    cat /etc/pgbackrest-secret/repo-cipher-pass
  )"
  exec pgbackrest \
    --config=/etc/pgbackrest/pgbackrest.conf \
    --stanza=postgres-ha --output=json info
' > evidence/full-backup-info.json
FULL_LABEL=$(jq -er '.[0].backup
| map(select(.type == "full")) | last | .label' \
evidence/full-backup-info.json)
printf '%s\n' "$FULL_LABEL" > evidence/full-label.txt

```

Expected: backup completes successfully and the recorded label ends with F. Record its start/stop time, database size, repository size and WAL start/stop. Do not assume the full backup contains rows inserted after it completed.

5.2 Inspect the GCS backup objects

```

gcloud storage ls \
  "gs://$BUCKET/pgbackrest/postgres-ha/backup/"
gcloud storage ls \
  "gs://$BUCKET/pgbackrest/postgres-ha/backup/postgres-ha/"
kubectl -n "$NS" exec -i "$PRIMARY" -c postgres -- \
runuser -u postgres -- psql -X -U postgres -d banking \
-v ON_ERROR_STOP=1 -c \
"INSERT INTO day4_lab.events(note) VALUES ('after full backup');"

```

Expect backup metadata and a directory for the full backup label. Encryption means objects are not readable as ordinary SQL or unencrypted data files. Use pgBackRest info and restore to interpret and recover them.

Lab 6 Incremental backup

Objective: capture changes since the previous backup and inspect dependency metadata.

```
PODS=$(kubectl -n "$NS" get pods -l cluster-name="$CLUSTER" \
-o jsonpath='{range .items[*]}{.metadata.name}{"\n"}{end}')
```

```
PRIMARY=""
PRIMARY_COUNT=0
for pod in $PODS; do
    ROLE=$(kubectl -n "$NS" exec "$pod" -c postgres -- \
        runuser -u postgres -- psql -X -U postgres -d postgres \
        -Atqc 'SELECT pg_is_in_recovery();') || exit 1
    if [ "$ROLE" = "f" ]; then
        PRIMARY="$pod"
        PRIMARY_COUNT=$((PRIMARY_COUNT + 1))
    fi
done
if [ "$PRIMARY_COUNT" -ne 1 ]; then
    echo "STOP: expected exactly one primary; found $PRIMARY_COUNT"
    exit 1
fi
export PRIMARY
printf 'Current primary: %s\n' "$PRIMARY"
kubectl -n "$NS" exec -i "$PRIMARY" -c postgres -- \
    runuser -u postgres -- psql -X -U postgres -d banking \
    -v ON_ERROR_STOP=1 -c \
    "INSERT INTO day4_lab.events(note)
    VALUES ('before incremental backup');"
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- \
    runuser -u postgres -- sh -ec '
    export PGBACKREST_REPO1_CIPHER_PASS="$(
        cat /etc/pgbackrest-secret/repo-cipher-pass
    )"
    exec pgbackrest \
        --config=/etc/pgbackrest/pgbackrest.conf \
        --stanza=postgres-ha --type=incr backup
    ,
    kubectl -n "$NS" exec "$PRIMARY" -c postgres -- \
    runuser -u postgres -- sh -ec '
    export PGBACKREST_REPO1_CIPHER_PASS="$(
        cat /etc/pgbackrest-secret/repo-cipher-pass
    )"
    exec pgbackrest \
        --config=/etc/pgbackrest/pgbackrest.conf \
        --stanza=postgres-ha info
    ,
    kubectl -n "$NS" exec "$PRIMARY" -c postgres -- \
    runuser -u postgres -- sh -ec '
    export PGBACKREST_REPO1_CIPHER_PASS="$(
        cat /etc/pgbackrest-secret/repo-cipher-pass
    )"
    ,
```

```
exec pgbackrest \  
  --config=/etc/pgbackrest/pgbackrest.conf \  
  --stanza=postgres-ha --output=json info  
' > evidence/incremental-info.json
```

Expected: newest backup label ends with I and refers to its full/preceding backups. An incremental backup is not independently restorable without its dependency chain. On a tiny database, metadata overhead can dominate; a small size difference does not mean the exercise failed.

Lab 7 Differential backup and retention

Objective: capture changes since the latest full backup and understand repository policies.

7.1 Differential backup

```
PODS=$(kubectl -n "$NS" get pods -l cluster-name="$CLUSTER" \
-o jsonpath='{range .items[*]}{.metadata.name}{"\n"}{end}')
PRIMARY=""
PRIMARY_COUNT=0
for pod in $PODS; do
    ROLE=$(kubectl -n "$NS" exec "$pod" -c postgres -- \
    runuser -u postgres -- psql -X -U postgres -d postgres \
    -Atqc 'SELECT pg_is_in_recovery();') || exit 1
    if [ "$ROLE" = "f" ]; then
        PRIMARY="$pod"
        PRIMARY_COUNT=$((PRIMARY_COUNT + 1))
    fi
done
if [ "$PRIMARY_COUNT" -ne 1 ]; then
    echo "STOP: expected exactly one primary; found $PRIMARY_COUNT"
    exit 1
fi
export PRIMARY
printf 'Current primary: %s\n' "$PRIMARY"
kubectl -n "$NS" exec -i "$PRIMARY" -c postgres -- \
    runuser -u postgres -- psql -X -U postgres -d banking \
    -v ON_ERROR_STOP=1 -c \
    "INSERT INTO day4_lab.events(note)
    VALUES ('before differential backup');"
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- \
    runuser -u postgres -- sh -ec '
    export PGBACKREST_REPO1_CIPHER_PASS="$(
    cat /etc/pgbackrest-secret/repo-cipher-pass
    )"
    exec pgbackrest \
        --config=/etc/pgbackrest/pgbackrest.conf \
        --stanza=postgres-ha --type=diff backup
    ,
    kubectl -n "$NS" exec "$PRIMARY" -c postgres -- \
    runuser -u postgres -- sh -ec '
    export PGBACKREST_REPO1_CIPHER_PASS="$(
    cat /etc/pgbackrest-secret/repo-cipher-pass
    )"
    exec pgbackrest \
        --config=/etc/pgbackrest/pgbackrest.conf \
        --stanza=postgres-ha info
    ,
    kubectl -n "$NS" exec "$PRIMARY" -c postgres -- \
    runuser -u postgres -- sh -ec '
    export PGBACKREST_REPO1_CIPHER_PASS="$(
```

```

    cat /etc/pgbackrest-secret/repo-cipher-pass
)"
exec pgbackrest \
  --config=/etc/pgbackrest/pgbackrest.conf \
  --stanza=postgres-ha --output=json info
' > evidence/differential-info.json
RESTORE_LABEL=$(jq -er '.[0].backup | last | .label' \
  evidence/differential-info.json)
printf '%s\n' "$RESTORE_LABEL" > evidence/restore-label.txt

```

Expected: newest label ends with D. This selected set will be restored in Lab 8 and should contain the baseline, incremental and differential marker rows. pgBackRest retrieves the required full backup automatically.

7.2 Compression and encryption

The configuration uses zst level 3 and AES-256-CBC repository encryption. Compression reduces stored bytes but adds CPU work; encryption protects repository contents but does not replace IAM, key management or TLS. Keep the original passphrase for as long as any encrypted backup is retained. Do not change cipher type on an existing repository as an experiment.

7.3 Retention preview and expiration

```

PODS=$(kubectl -n "$NS" get pods -l cluster-name="$CLUSTER" \
  -o jsonpath='{range .items[*]}{.metadata.name}{ "\n" }{end}')
PRIMARY=""
PRIMARY_COUNT=0
for pod in $PODS; do
  ROLE=$(kubectl -n "$NS" exec "$pod" -c postgres -- \
    runuser -u postgres -- psql -X -U postgres -d postgres \
    -Atqc 'SELECT pg_is_in_recovery();') || exit 1
  if [ "$ROLE" = "f" ]; then
    PRIMARY="$pod"
    PRIMARY_COUNT=$((PRIMARY_COUNT + 1))
  fi
done
if [ "$PRIMARY_COUNT" -ne 1 ]; then
  echo "STOP: expected exactly one primary; found $PRIMARY_COUNT"
  exit 1
fi
export PRIMARY
printf 'Current primary: %s\n' "$PRIMARY"
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- \
  runuser -u postgres -- sh -ec '
    export PGBACKREST_REPO1_CIPHER_PASS=$(
      cat /etc/pgbackrest-secret/repo-cipher-pass
    )"
    exec pgbackrest \
      --config=/etc/pgbackrest/pgbackrest.conf \
      --stanza=postgres-ha info

```

```
,
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- \
  runuser -u postgres -- sh -ec '
    export PGBACKREST_REPO1_CIPHER_PASS="$(
      cat /etc/pgbackrest-secret/repo-cipher-pass
    )"
    exec pgbackrest \
      --config=/etc/pgbackrest/pgbackrest.conf \
      --stanza=postgres-ha --dry-run expire
  '
,
```

The repository keeps two full backups and up to four differential backups subject to dependency rules. Incremental backups remain tied to their parents. Retention counts are not guarantees of a fixed number of days. Automatic expiration can run after successful backups; record the labels needed for a drill before taking many additional full backups.

Only when the preview is understood and no required recovery set will be lost, run:

```
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- \
  runuser -u postgres -- sh -ec '
    export PGBACKREST_REPO1_CIPHER_PASS="$(
      cat /etc/pgbackrest-secret/repo-cipher-pass
    )"
    exec pgbackrest \
      --config=/etc/pgbackrest/pgbackrest.conf \
      --stanza=postgres-ha expire
  '
,
```

```
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- \
  runuser -u postgres -- sh -ec '
    export PGBACKREST_REPO1_CIPHER_PASS="$(
      cat /etc/pgbackrest-secret/repo-cipher-pass
    )"
    exec pgbackrest \
      --config=/etc/pgbackrest/pgbackrest.conf \
      --stanza=postgres-ha info
  '
,
```

Never delete individual GCS backup or WAL objects manually to demonstrate retention. Do not expire backups while a restore drill depends on them. Object-store soft delete, versioning or retention locks can affect when billable storage is actually released.

Lab 8 Complete physical restore to independent storage

Objective: restore a selected physical backup, start PostgreSQL, and verify recovered data without operator reconciliation or live-PVC conflicts.

This replaces the incomplete recovery placeholders in earlier manuals. A new standalone Pod runs the same image as the source with its entrypoint overridden. No Patroni process is started. A new PVC holds PGDATA and remapped tablespaces. Recovery listens only on loopback and uses a local socket; no Service exposes it to applications.

8.1 Discover image identity and PostgreSQL binary directory

```
PODS=$(kubectl -n "$NS" get pods -l cluster-name="$CLUSTER" \
-o jsonpath='{range .items[*]}{.metadata.name}{"\n"}{end}')
```

```
PRIMARY=""
PRIMARY_COUNT=0
for pod in $PODS; do
  ROLE=$(kubectl -n "$NS" exec "$pod" -c postgres -- \
    runuser -u postgres -- psql -X -U postgres -d postgres \
    -Atqc 'SELECT pg_is_in_recovery();') || exit 1
  if [ "$ROLE" = "f" ]; then
    PRIMARY="$pod"
    PRIMARY_COUNT=$((PRIMARY_COUNT + 1))
  fi
done
if [ "$PRIMARY_COUNT" -ne 1 ]; then
  echo "STOP: expected exactly one primary; found $PRIMARY_COUNT"
  exit 1
fi
export PRIMARY
printf 'Current primary: %s\n' "$PRIMARY"
REC_IMAGE=$(kubectl -n "$NS" get pod "$PRIMARY" \
-o jsonpath='{.spec.containers[?(@.name=="postgres")].image}')
```

```
PG_UID=$(kubectl -n "$NS" exec -i "$PRIMARY" -c postgres -- \
  runuser -u postgres -- id -u)
PG_GID=$(kubectl -n "$NS" exec -i "$PRIMARY" -c postgres -- \
  runuser -u postgres -- id -g)
PG_BINDIR=$(kubectl -n "$NS" exec -i "$PRIMARY" -c postgres -- \
  runuser -u postgres -- sh -ec 'command -v pg_ctl' \
  | xargs dirname)
export REC_IMAGE PG_UID PG_GID PG_BINDIR
kubectl -n "$NS" exec -i "$PRIMARY" -c postgres -- \
  runuser -u postgres -- "$PG_BINDIR/postgres" --version
printf 'Image: %s\nUID: %s GID: %s\nBinaries: %s\n' \
  "$REC_IMAGE" "$PG_UID" "$PG_GID" "$PG_BINDIR"
kubectl get storageclass
kubectl -n "$NS" exec -i "$PRIMARY" -c postgres -- \
  runuser -u postgres -- psql -X -U postgres -d postgres -Atqc \
```

```

"SELECT name || ' = ' || setting FROM pg_settings
WHERE name IN ('max_connections','max_worker_processes',
               'max_wal_senders','max_prepared_transactions',
               'max_locks_per_transaction') ORDER BY name;" \
> evidence/recovery-minimums.conf
cat evidence/recovery-minimums.conf
kubectl -n "$NS" exec -i "$PRIMARY" -c postgres -- \
  runuser -u postgres -- du -sh "$PGDATA_ACTUAL"
kubectl -n "$NS" exec -i "$PRIMARY" -c postgres -- \
  runuser -u postgres -- psql -X -U postgres -d postgres -c \
  'SELECT spcname, pg_size_pretty(pg_tablespace_size(oid))
  FROM pg_tablespace;'

```

Expected binary major version: 18. If pg_ctl is not on PATH, use /usr/lib/postgresql/18/bin only after verifying that path exists and postgres --version reports 18. The recorded connection/worker limits are retained during replay because recovery can fail when they are below source values. Do not tune these source settings during the drill. Size the new PVC for the whole restored cluster, WAL and external tablespaces, with headroom. The 20Gi value below is a small-lab default, not an estimate of your database size.

```

REC_SIZE=20Gi
read -r -p 'StorageClass name from the list: ' REC_SC
export REC_SIZE REC_SC
test -n "$REC_SC"
kubectl get storageclass "$REC_SC"
POD_SA=$(kubectl -n "$NS" get pod "$PRIMARY" \
  -o jsonpath='{.spec.serviceAccountName}')
export POD_SA
kubectl -n "$NS" get pod "$PRIMARY" -o json \
  > evidence/recovery-source-pod.json

```

Private registry users: refresh the pull Secret from 4.5 if its token has expired. Recovery copies the actual pod's imagePullSecrets; if your source pod predates the ServiceAccount change, ensure the chosen ServiceAccount also has the pull Secret.

8.2 Create the complete manifest generator

This file creates one PVC and one standalone Pod for the name passed to it. It mounts all three backup resources and an emptyDir for socket and temporary files. fsGroup is a shared group for volume access; it is not an arbitrary value like 10001. UID and GID are discovered from the actual image.

```

cat > make-recovery.py <<'PY'
import json, os, sys
name=sys.argv[1]
if name not in ('day4-full-recovery','day4-pitr-recovery'):
    raise SystemExit('Unexpected recovery name')
e=os.environ
uid, gid=int(e['PG_UID']), int(e['PG_GID'])
src=json.load(open('evidence/recovery-source-pod.json'))

```

```

pvc={'apiVersion':'v1','kind':'PersistentVolumeClaim',
  'metadata':{'name':name+'-data','namespace':e['NS']},
  'spec':{'accessModes':['ReadWriteOnce'],
    'storageClassName':e['REC_SC'],
    'resources':{'requests':{'storage':e['REC_SIZE']}}}}
pod={'apiVersion':'v1','kind':'Pod',
  'metadata':{'name':name,'namespace':e['NS'],
    'labels':{'app':name}},
  'spec':{'
    'restartPolicy':'Never',
    'automountServiceAccountToken':False,
    'serviceAccountName':e['POD_SA'],
    'securityContext':{'runAsUser':uid,'runAsGroup':gid,
      'fsGroup':gid},
    'containers':[{
      'name':'postgres','image':e['REC_IMAGE'],
      'command':['sh','-ec','sleep infinity'],
      'env':[{'name':'PG_BINDIR','value':e['PG_BINDIR']}],
      'securityContext':{'allowPrivilegeEscalation':False},
      'volumeMounts':[
        {'name':'data','mountPath':'/restore'},
        {'name':'tmp','mountPath':'/tmp'},
        {'name':'config','mountPath':'/etc/pgbackrest',
          'readOnly':True},
        {'name':'cipher','mountPath':'/etc/pgbackrest-secret',
          'readOnly':True},
        {'name':'gcs','mountPath':'/etc/pgbackrest-gcs',
          'readOnly':True}]]},
    'volumes':[
      {'name':'data','persistentVolumeClaim':{'
        'claimName':name+'-data'}},
      {'name':'tmp','emptyDir':{}},
      {'name':'config','configMap':{'name':'pgbackrest-config'}},
      {'name':'cipher','secret':{'
        'secretName':'pgbackrest-repo-secret'}},
      {'name':'gcs','secret':{'secretName':'pgbackrest-gcs-key'}}]]}
pulls=src['spec'].get('imagePullSecrets',[])
if pulls:
    pod['spec']['imagePullSecrets']=pulls
print(json.dumps({'apiVersion':'v1','kind':'List',
  'items':[pvc,pod]}, indent=2))
PY
python3 make-recovery.py day4-full-recovery \
  > manifests/day4-full-recovery.json
python3 -m json.tool manifests/day4-full-recovery.json \
  > /dev/null
kubectl apply --dry-run=server \
  -f manifests/day4-full-recovery.json
kubectl apply -f manifests/day4-full-recovery.json
kubectl -n "$NS" wait --for=condition=Ready \
  pod/day4-full-recovery --timeout=10m
kubectl -n "$NS" get pvc day4-full-recovery-data

```

Expected: PVC Bound and Pod Ready. Ready here means the toolbox is running, not that PostgreSQL has been recovered. A WaitForFirstConsumer StorageClass may bind only after the pod is scheduled.

8.3 Verify isolation and write permissions before restore

```
REC=day4-full-recovery
kubectl -n "$NS" get pod "$REC" -o json \
  | jq '.spec.volumes'
kubectl -n "$NS" exec -i "$REC" -c postgres -- \
  sh -ec '
  id
  test -r /etc/pgbackrest/pgbackrest.conf
  test -r /etc/pgbackrest-secret/repo-cipher-pass
  test -r /etc/pgbackrest-gcs/gcs-key.json
  mkdir -p /restore/pgdata /restore/tablespaces
  chmod 700 /restore/pgdata
  touch /restore/write-test
  rm /restore/write-test
  test -z "$(ls -A /restore/pgdata)"
  echo "New recovery PGDATA is empty and writable"
  '
```

Confirm the only data claim is day4-full-recovery-data. If directory creation or touch fails, stop and use the permission repair below. A permission failure is not fixed by changing pgBackRest stanza names.

8.4 Permission repair only if 8.3 failed

Some local storage drivers do not apply fsGroup as expected. Add a root init container on this NEW recovery Pod to initialize ownership of this NEW PVC. This does not change the source image user or source PVC permissions. A restricted cluster may prohibit root init containers; use the storage administrator's approved ownership mechanism in that case.

```
python3 - <<'PY'
import json, os
path='manifests/day4-full-recovery.json'
d=json.load(open(path))
pod=d['items'][1]
uid, gid=os.environ['PG_UID'], os.environ['PG_GID']
pod['spec']['initContainers']=[{
  'name':'prepare-recovery-permissions',
  'image':os.environ['REC_IMAGE'],
  'command':['sh','-ec',
    'mkdir -p /restore/pgdata /restore/tablespaces; '
    f'chown -R {uid}:{gid} /restore; '
    'chmod 700 /restore/pgdata'],
  'securityContext':{'runAsUser':0,'runAsGroup':0},
  'volumeMounts':[{'name':'data','mountPath':'/restore'}]]]
json.dump(d,open(path,'w'),indent=2)
PY
```

```
kubectl -n "$NS" delete pod day4-full-recovery --wait=true
kubectl apply -f manifests/day4-full-recovery.json
kubectl -n "$NS" wait --for=condition=Ready \
  pod/day4-full-recovery --timeout=10m
```

Repeat 8.3. Do not delete the PVC to fix permissions. Kubernetes generally cannot patch immutable pod fields such as volume definitions; edit the saved manifest, recreate only the recovery pod, and retain its intended PVC.

8.5 Restore the recorded backup set

```
RESTORE_LABEL=$(cat evidence/restore-label.txt)
test -n "$RESTORE_LABEL"
kubectl -n "$NS" exec -i "$REC" -c postgres -- \
  env RESTORE_LABEL="$RESTORE_LABEL" sh -ec '
  export PGBACKREST_REP01_CIPHER_PASS="$(
    cat /etc/pgbackrest-secret/repo-cipher-pass
  )"
  pgbackrest --config=/etc/pgbackrest/pgbackrest.conf \
    --stanza=postgres-ha --pg1-path=/restore/pgdata \
    --tablespace-map-all=/restore/tablespaces \
    --set="$RESTORE_LABEL" --type=immediate \
    --target-action=promote restore
  '
```

Expected: restore command completes successfully. `--type=immediate` recovers this selected backup to its earliest consistent state, making row verification deterministic. To restore the latest eligible backup, omit `--set`; the literal `--set=latest` is not a valid backup label. To replay all available WAL instead, use `--type=default` and omit recovery target options; that is a different recovery objective. Keep this exercise's immediate target for predictable verification.

The restore command writes data files and recovery settings. PostgreSQL still needs to start to replay WAL. Restored file contents alone do not mean recovery is complete.

8.6 Create the isolated PostgreSQL startup script

Spilo data files can reference Patroni configuration, preload libraries and source paths. For the isolated drill, use a minimal configuration, preserving only pgBackRest-generated recovery settings from `postgresql.auto.conf`. The same PostgreSQL image retains installed libraries; extensions that require preload may need additional reviewed settings before application-level tests.

```
cat > start-recovery.sh <<'SH'
#!/bin/sh
set -eu
DATA=/restore/pgdata
test -s "$DATA/PG_VERSION"
test "$(cat "$DATA/PG_VERSION")" = "18"
test ! -e "$DATA/postmaster.pid"
cp "$DATA/postgresql.auto.conf" \
```

```

/restore/postgresql.auto.conf.restored
# Keep generated archive recovery settings, discard source overrides.
sed -n -E \
  '/^(restore_command|recovery_target[^ ]*)[[[:space:]]*=/p' \
  /restore/postgresql.auto.conf.restored \
  > "$DATA/postgresql.auto.conf"
grep -q '^restore_command' "$DATA/postgresql.auto.conf"
cat > "$DATA/postgresql.conf" <<'CONF'
listen_addresses = '127.0.0.1'
port = 5432
unix_socket_directories = '/tmp'
ssl = off
archive_mode = off
archive_command = ''
archive_library = ''
shared_preload_libraries = ''
logging_collector = off
hot_standby = on
CONF
test -s /restore/recovery-minimums.conf
cat /restore/recovery-minimums.conf >> "$DATA/postgresql.conf"
cat > "$DATA/pg_hba.conf" <<'HBA'
local all postgres trust
local all all reject
host all all 127.0.0.1/32 reject
host all all ::1/128 reject
HBA
: > "$DATA/pg_ident.conf"
chmod 600 "$DATA/postgresql.conf" "$DATA/pg_hba.conf"
export PGBACKREST_REP01_CIPHER_PASS="$(
  cat /etc/pgbackrest-secret/repo-cipher-pass
)"
"$PG_BINDIR/pg_ctl" -D "$DATA" \
  -l /restore/postgresql.log -w -t 300 start
SH
cat evidence/recovery-minimums.conf | kubectl -n "$NS" exec -i "$REC" -c postgres -- \
  \
  sh -ec 'cat > /restore/recovery-minimums.conf'
cat start-recovery.sh | kubectl -n "$NS" exec -i "$REC" -c postgres -- \
  sh

```

Trust is restricted to local socket access as the postgres database user inside this isolated lab pod. No application Service is created. Do not reuse this HBA file in the live cluster.

8.7 Validate the recovered database

```
kubectl -n "$NS" exec -i "$REC" -c postgres -- \
  "$PG_BINDIR/psql" -X -h /tmp \
  -U postgres -d postgres <<'SQL'
\set ON_ERROR_STOP on
SELECT pg_is_in_recovery(), now();
SHOW data_directory;
SHOW archive_mode;
SELECT timeline_id FROM pg_control_checkpoint();
SQL
kubectl -n "$NS" exec -i "$REC" -c postgres -- \
  "$PG_BINDIR/psql" -X -h /tmp \
  -U postgres -d banking -c \
  'SELECT * FROM day4_lab.events ORDER BY id;'
kubectl -n "$NS" exec -i "$REC" -c postgres -- \
  tail -n 60 /restore/postgresql.log
```

Expected: `pg_is_in_recovery()` is f, PGDATA is `/restore/pgdata`, `archive_mode` is off, and rows through the selected differential backup are present. The new timeline reflects promotion after recovery. The live cluster remains independently writable. A physical restore contains all databases and global objects in the selected cluster backup.

8.8 Stop the recovered instance cleanly

```
kubectl -n "$NS" exec -i "$REC" -c postgres -- \
  "$PG_BINDIR/pg_ctl" \
  -D /restore/pgdata -m fast -w stop
```

Keep this recovery PVC as evidence while completing PITR on a different PVC. If the toolbox pod is recreated later, do not rerun restore into nonempty data. To restart an already recovered server, use `pg_ctl` with the same config and exported cipher environment; do not run `start-recovery.sh` again because it is a first-start preparation script.

Lab 9 Complete point in time recovery

Objective: prove recovery of committed good data while excluding a later destructive transaction. All destructive SQL is restricted to the dedicated day4_lab.pitr_events table.

9.1 Prepare the table and take a new full backup

```
PODS=$(kubectl -n "$NS" get pods -l cluster-name="$CLUSTER" \
-o jsonpath='{range .items[*]}{.metadata.name}{ "\n"}{end}')
PRIMARY=""
PRIMARY_COUNT=0
for pod in $PODS; do
    ROLE=$(kubectl -n "$NS" exec "$pod" -c postgres -- \
    runuser -u postgres -- psql -X -U postgres -d postgres \
    -Atqc 'SELECT pg_is_in_recovery();') || exit 1
    if [ "$ROLE" = "f" ]; then
        PRIMARY="$pod"
        PRIMARY_COUNT=$((PRIMARY_COUNT + 1))
    fi
done
if [ "$PRIMARY_COUNT" -ne 1 ]; then
    echo "STOP: expected exactly one primary; found $PRIMARY_COUNT"
    exit 1
fi
export PRIMARY
printf 'Current primary: %s\n' "$PRIMARY"
kubectl -n "$NS" exec -i "$PRIMARY" -c postgres -- \
    runuser -u postgres -- psql -X -U postgres -d banking <<'SQL'
\set ON_ERROR_STOP on
CREATE TABLE IF NOT EXISTS day4_lab.pitr_events (
    id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    note text NOT NULL,
    created_at timestamptz DEFAULT clock_timestamp()
);
INSERT INTO day4_lab.pitr_events(note) VALUES ('PITR baseline');
SQL
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- \
    runuser -u postgres -- sh -ec '
        export PGBACKREST_REPO1_CIPHER_PASS="$(
            cat /etc/pgbackrest-secret/repo-cipher-pass
        )"
        exec pgbackrest \
            --config=/etc/pgbackrest/pgbackrest.conf \
            --stanza=postgres-ha --type=full backup
    '
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- \
    runuser -u postgres -- sh -ec '
        export PGBACKREST_REPO1_CIPHER_PASS="$(
            cat /etc/pgbackrest-secret/repo-cipher-pass
        )"
        exec pgbackrest \
```

```

    --config=/etc/pgbackrest/pgbackrest.conf \
    --stanza=postgres-ha --output=json info
' > evidence/pitr-backup-info.json
PITR_LABEL=$(jq -er '.[0].backup
| map(select(.type == "full")) | last | .label' \
evidence/pitr-backup-info.json)
printf '%s\n' "$PITR_LABEL" > evidence/pitr-label.txt
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- \
runuser -u postgres -- sh -ec '
    export PGBACKREST_REPO1_CIPHER_PASS="$(
        cat /etc/pgbackrest-secret/repo-cipher-pass
    )"
    exec pgbackrest \
        --config=/etc/pgbackrest/pgbackrest.conf \
        --stanza=postgres-ha info
'

```

This backup must finish before the target time. No later backup is substituted merely because it is newer. Ensure the chosen full label is retained until validation completes.

9.2 Commit a good row and record the server UTC time

```

kubectl -n "$NS" exec -i "$PRIMARY" -c postgres -- \
runuser -u postgres -- psql -X -U postgres -d banking \
-v ON_ERROR_STOP=1 -c \
"INSERT INTO day4_lab.pitr_events(note) VALUES ('PITR good row');"
TARGET_TIME=$(kubectl -n "$NS" exec -i "$PRIMARY" -c postgres -- \
runuser -u postgres -- psql -X -U postgres \
-d postgres -Atqc \
"SELECT to_char(clock_timestamp() AT TIME ZONE 'UTC',
                'YYYY-MM-DD HH24:MI:SS.US') || '+00';")
printf '%s\n' "$TARGET_TIME" > evidence/pitr-target-utc.txt
kubectl -n "$NS" exec -i "$PRIMARY" -c postgres -- \
runuser -u postgres -- psql -X -U postgres -d banking -Atqc \
'SELECT count(*) FROM day4_lab.pitr_events;' \
> evidence/pitr-expected-count.txt
printf 'Target UTC: %s\n' "$TARGET_TIME"
sleep 3

```

The INSERT ran as a separate committed transaction before the timestamp was captured. The delay makes the later DELETE clearly distinguishable. The explicit +00 avoids workstation timezone ambiguity.

9.3 Simulate the mistake and archive the WAL through it

```
kubectl -n "$NS" exec -i "$PRIMARY" -c postgres -- \
  runuser -u postgres -- psql -X -U postgres -d banking <<'SQL'
\set ON_ERROR_STOP on
DELETE FROM day4_lab.pitr_events;
SELECT count(*) AS rows_after_mistake FROM day4_lab.pitr_events;
SELECT pg_switch_wal();
SQL
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- \
  runuser -u postgres -- sh -ec '
    export PGBACKREST_REP01_CIPHER_PASS="$(
      cat /etc/pgbackrest-secret/repo-cipher-pass
    )"
    exec pgbackrest \
      --config=/etc/pgbackrest/pgbackrest.conf \
      --stanza=postgres-ha check
  '
kubectl -n "$NS" exec -i "$PRIMARY" -c postgres -- \
  runuser -u postgres -- psql -X -U postgres -d postgres -x -c \
  'SELECT archived_count, failed_count, last_archived_wal,
    last_archived_time FROM pg_stat_archiver;'
```

Expected: source count is zero and pgBackRest check succeeds. WAL after the target, including the unwanted DELETE, must be available so recovery can encounter a transaction beyond the target and stop correctly. This does not mean that DELETE will be applied on the recovery target.

9.4 Create a separate PITR PVC and Pod

Use the generator created in 8.2. It contains the complete mounts, image, ownership and storage settings. It creates a different data claim from Lab 8.

```
python3 make-recovery.py day4-pitr-recovery \
  > manifests/day4-pitr-recovery.json
kubectl apply --dry-run=server \
  -f manifests/day4-pitr-recovery.json
kubectl apply -f manifests/day4-pitr-recovery.json
kubectl -n "$NS" wait --for=condition=Ready \
  pod/day4-pitr-recovery --timeout=10m
REC=day4-pitr-recovery
kubectl -n "$NS" exec -i "$REC" -c postgres -- \
  sh -ec '
    mkdir -p /restore/pgdata /restore/tablespaces
    chmod 700 /restore/pgdata
    touch /restore/write-test
    rm /restore/write-test
    test -z "$(ls -A /restore/pgdata)"
  '
,
```

If your storage driver required the init-container fix in 8.4, copy that approved init container from the full recovery manifest into this manifest before recreating this Pod:

```
python3 - <<'PY'
import json
src=json.load(open('manifests/day4-full-recovery.json'))
path='manifests/day4-pitr-recovery.json'
dst=json.load(open(path))
init=src['items'][1]['spec'].get('initContainers')
if not init:
    raise SystemExit('No ownership fix recorded; inspect permissions')
dst['items'][1]['spec']['initContainers']=init
json.dump(dst,open(path,'w'),indent=2)
PY
kubectl -n "$NS" delete pod day4-pitr-recovery --wait=true
kubectl apply -f manifests/day4-pitr-recovery.json
kubectl -n "$NS" wait --for=condition=Ready \
    pod/day4-pitr-recovery --timeout=10m
```

Run that repair block only on permission failure, then repeat the write/empty-directory checks.

9.5 Restore to the recorded timestamp

```
PITR_LABEL=$(cat evidence/pitr-label.txt)
TARGET_TIME=$(cat evidence/pitr-target-utc.txt)
test -n "$PITR_LABEL" && test -n "$TARGET_TIME"
kubectl -n "$NS" exec -i "$REC" -c postgres -- \
    env PITR_LABEL="$PITR_LABEL" \
    TARGET_TIME="$TARGET_TIME" sh -ec '
    export PGBACKREST_REPO1_CIPHER_PASS="$(
        cat /etc/pgbackrest-secret/repo-cipher-pass
    )"
    pgbackrest --config=/etc/pgbackrest/pgbackrest.conf \
        --stanza=postgres-ha --pg1-path=/restore/pgdata \
        --tablespace-map-all=/restore/tablespaces \
        --set="$PITR_LABEL" --type=time \
        --target="$TARGET_TIME" --target-action=promote restore
    '
```

The target is passed into the container with env, so it is not lost between workstation and pod shells. This restores a complete physical cluster and replays WAL to the target, not just one table.

9.6 Start PostgreSQL and verify the target was reached

```
cat evidence/recovery-minimums.conf | kubectl -n "$NS" exec -i "$REC" -c postgres -- \
\
sh -ec 'cat > /restore/recovery-minimums.conf'
cat start-recovery.sh | kubectl -n "$NS" exec -i "$REC" -c postgres -- \
sh
kubectl -n "$NS" exec -i "$REC" -c postgres -- \
"$PG_BINDIR/psql" -X -h /tmp \
-U postgres -d banking -c \
'SELECT * FROM day4_lab.pitr_events ORDER BY id;'
ACTUAL_COUNT=$(kubectl -n "$NS" exec -i "$REC" -c postgres -- \
"$PG_BINDIR/psql" -X -h /tmp \
-U postgres -d banking -Atqc \
'SELECT count(*) FROM day4_lab.pitr_events;')
EXPECTED_COUNT=$(cat evidence/pitr-expected-count.txt)
printf 'Expected %s; recovered %s\n' \
"$EXPECTED_COUNT" "$ACTUAL_COUNT"
test "$ACTUAL_COUNT" = "$EXPECTED_COUNT"
kubectl -n "$NS" exec -i "$REC" -c postgres -- \
"$PG_BINDIR/psql" -X -h /tmp \
-U postgres -d postgres -c \
'SELECT pg_is_in_recovery();
SELECT timeline_id FROM pg_control_checkpoint;')
kubectl -n "$NS" exec -i "$REC" -c postgres -- \
tail -n 100 /restore/postgresql.log
```

Expected: PITR good row is present, recovered count matches the pre-DELETE count, and `pg_is_in_recovery()` is f after promotion. Logs show recovery stopped at the configured target. The source table still has zero rows, demonstrating an independent recovery rather than source repair.

If logs say recovery ended before the configured recovery target was reached, do not remove the target to force startup. Check the timestamp, selected backup start/stop and complete WAL sequence. If a failover occurred during this exercise, record the source timeline and verify the relevant timeline history is archived before retrying.

9.7 Optional object recovery back into the live lab

A production cutover is outside this drill. To demonstrate restoring just the recovered table without overwriting the source, dump the recovered training table and load it into a NEW database on the live lab cluster.

```
kubectl -n "$NS" exec -i "$REC" -c postgres -- \
"$PG_BINDIR/pg_dump" -h /tmp -U postgres \
-d banking -Fc -t day4_lab.pitr_events \
> backups/logical/pitr-recovered-table.dump
PODS=$(kubectl -n "$NS" get pods -l cluster-name="$CLUSTER" \
-o jsonpath='{range .items[*]}{.metadata.name}{"\n"}{end}')
PRIMARY=""
PRIMARY_COUNT=0
```

```

for pod in $PODS; do
  ROLE=$(kubectl -n "$NS" exec "$pod" -c postgres -- \
    runuser -u postgres -- psql -X -U postgres -d postgres \
    -Atqc 'SELECT pg_is_in_recovery();') || exit 1
  if [ "$ROLE" = "f" ]; then
    PRIMARY="$pod"
    PRIMARY_COUNT=$((PRIMARY_COUNT + 1))
  fi
done
if [ "$PRIMARY_COUNT" -ne 1 ]; then
  echo "STOP: expected exactly one primary; found $PRIMARY_COUNT"
  exit 1
fi
export PRIMARY
printf 'Current primary: %s\n' "$PRIMARY"
PITR_COPY_DB="pitr_copy_$(date -u +%Y%m%d%H%M%S)"
kubectl -n "$NS" exec -i "$PRIMARY" -c postgres -- \
  runuser -u postgres -- createdb -U postgres "$PITR_COPY_DB"
kubectl -n "$NS" exec -i "$PRIMARY" -c postgres -- \
  runuser -u postgres -- psql -X -U postgres -d "$PITR_COPY_DB" \
  -v ON_ERROR_STOP=1 -c 'CREATE SCHEMA day4_lab;'
cat backups/logical/pitr-recovered-table.dump \
  | kubectl -n "$NS" exec -i "$PRIMARY" -c postgres -- \
  runuser -u postgres -- pg_restore -U postgres -d "$PITR_COPY_DB" \
  --no-owner --no-privileges --exit-on-error
kubectl -n "$NS" exec -i "$PRIMARY" -c postgres -- \
  runuser -u postgres -- psql -X -U postgres -d "$PITR_COPY_DB" \
  -c 'TABLE day4_lab.pitr_events;'
printf '%s\n' "$PITR_COPY_DB" > evidence/pitr-copy-db.txt

```

A table-selective dump does not automatically include all dependent objects; this controlled table uses only built-in types. Validate application dependencies separately for real object recovery.

Operations and troubleshooting

Primary changes and backup scheduling

Always run the explicit primary-discovery block before stanza-create, check, backup and archiver validation. The Service follows the current primary; a saved pod variable does not. An in-progress backup can fail during switchover; inspect pgBackRest info and retry from the new primary after archiving works. Do not assume two independent backup processes on two primaries are safe.

For a planned lab switchover, ensure all mounts and pgBackRest versions match, then use the interactive Patroni command. Select a healthy replica rather than killing a pod to force an election.

```
PODS=$(kubectl -n "$NS" get pods -l cluster-name="$CLUSTER" \
-o jsonpath='{range .items[*]}{.metadata.name}{"\n"}{end}')
PRIMARY=""
PRIMARY_COUNT=0
for pod in $PODS; do
  ROLE=$(kubectl -n "$NS" exec "$pod" -c postgres -- \
  runuser -u postgres -- psql -X -U postgres -d postgres \
  -Atqc 'SELECT pg_is_in_recovery();') || exit 1
  if [ "$ROLE" = "f" ]; then
    PRIMARY="$pod"
    PRIMARY_COUNT=$((PRIMARY_COUNT + 1))
  fi
done
if [ "$PRIMARY_COUNT" -ne 1 ]; then
  echo "STOP: expected exactly one primary; found $PRIMARY_COUNT"
  exit 1
fi
export PRIMARY
printf 'Current primary: %s\n' "$PRIMARY"
kubectl -n "$NS" exec -i "$PRIMARY" -c postgres -- \
  runuser -u postgres -- patronictl list
kubectl -n "$NS" exec -it "$PRIMARY" -c postgres -- \
  patronictl switchover "$CLUSTER"
PODS=$(kubectl -n "$NS" get pods -l cluster-name="$CLUSTER" \
-o jsonpath='{range .items[*]}{.metadata.name}{"\n"}{end}')
PRIMARY=""
PRIMARY_COUNT=0
for pod in $PODS; do
  ROLE=$(kubectl -n "$NS" exec "$pod" -c postgres -- \
  runuser -u postgres -- psql -X -U postgres -d postgres \
  -Atqc 'SELECT pg_is_in_recovery();') || exit 1
  if [ "$ROLE" = "f" ]; then
    PRIMARY="$pod"
    PRIMARY_COUNT=$((PRIMARY_COUNT + 1))
  fi
done
```

```

if [ "$PRIMARY_COUNT" -ne 1 ]; then
    echo "STOP: expected exactly one primary; found $PRIMARY_COUNT"
    exit 1
fi
export PRIMARY
printf 'Current primary: %s\n' "$PRIMARY"
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- \
    runuser -u postgres -- sh -ec '
        export PGBACKREST_REPO1_CIPHER_PASS="$(
            cat /etc/pgbackrest-secret/repo-cipher-pass
        )"
        exec pgbackrest \
            --config=/etc/pgbackrest/pgbackrest.conf \
            --stanza=postgres-ha check
    '
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- \
    runuser -u postgres -- sh -ec '
        export PGBACKREST_REPO1_CIPHER_PASS="$(
            cat /etc/pgbackrest-secret/repo-cipher-pass
        )"
        exec pgbackrest \
            --config=/etc/pgbackrest/pgbackrest.conf \
            --stanza=postgres-ha info
    '

```

If the Patroni scope displayed by `patronictl list` differs from the CR name, use the actual scope in `patronictl` commands. After switchover, inspect the new primary's archive settings and verify another backup. Pausing Patroni pauses automatic management; it does not pause PostgreSQL WAL replay or repair deleted data.

Daily validation commands

```
PODS=$(kubectl -n "$NS" get pods -l cluster-name="$CLUSTER" \
-o jsonpath='{range .items[*]}{.metadata.name}{"\n"}{end}')
PRIMARY=""
PRIMARY_COUNT=0
for pod in $PODS; do
    ROLE=$(kubectl -n "$NS" exec "$pod" -c postgres -- \
    runuser -u postgres -- psql -X -U postgres -d postgres \
    -Atqc 'SELECT pg_is_in_recovery();') || exit 1
    if [ "$ROLE" = "f" ]; then
        PRIMARY="$pod"
        PRIMARY_COUNT=$((PRIMARY_COUNT + 1))
    fi
done
if [ "$PRIMARY_COUNT" -ne 1 ]; then
    echo "STOP: expected exactly one primary; found $PRIMARY_COUNT"
    exit 1
fi
export PRIMARY
printf 'Current primary: %s\n' "$PRIMARY"
kubectl -n "$NS" exec -i "$PRIMARY" -c postgres -- \
    runuser -u postgres -- patronictl list
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- \
    runuser -u postgres -- sh -ec '
    export PGBACKREST_REPO1_CIPHER_PASS=$(
        cat /etc/pgbackrest-secret/repo-cipher-pass
    )"
    exec pgbackrest \
        --config=/etc/pgbackrest/pgbackrest.conf \
        --stanza=postgres-ha check
'
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- \
    runuser -u postgres -- sh -ec '
    export PGBACKREST_REPO1_CIPHER_PASS=$(
        cat /etc/pgbackrest-secret/repo-cipher-pass
    )"
    exec pgbackrest \
        --config=/etc/pgbackrest/pgbackrest.conf \
        --stanza=postgres-ha info
'
kubectl -n "$NS" exec -i "$PRIMARY" -c postgres -- \
    runuser -u postgres -- psql -X -U postgres -d postgres -x -c \
    'SELECT * FROM pg_stat_archiver;'
kubectl -n "$NS" logs "$PRIMARY" -c postgres --tail=150
kubectl -n "$NS" get events --sort-by=.metadata.creationTimestamp
```

When logging_collector writes PostgreSQL logs into files instead of stderr, discover them from PostgreSQL rather than assuming kubectl logs contains every database message:

```
kubectl -n "$NS" exec -i "$PRIMARY" -c postgres -- \
    runuser -u postgres -- psql -X -U postgres -d postgres -c \
```

```
'SHOW logging_collector; SHOW log_directory;
SHOW log_filename; SELECT pg_current_logfile();'
```

Use the returned filename with tail inside the pod. A NULL pg_current_logfile can be normal when logging_collector is off. For recovery Pods, PostgreSQL output is /restore/postgresql.log, while kubectl logs may be empty because the main container command is sleep.

Troubleshooting by symptom

Missing binary or unsupported PostgreSQL version: inspect pgbackrest version on every pod. Complete the immutable image branch; do not apt install interactively and assume it survives restart.

ImagePullBackOff: inspect kubectl describe pod and its events. Confirm the image tag/digest exists, registry IAM allows pulling and imagePullSecrets is attached. Refresh short-lived registry tokens. A GCS storage credential is not automatically an Artifact Registry pull credential.

metadata.google.internal not found: the mounted pgbackrest.conf still has key-type=auto or an older config is selected. Confirm key-type=service and /etc/pgbackrest-gcs/gcs-key.json inside the pod.

repo1-cipher-pass missing: load PGBACKREST_REPO1_CIPHER_PASS inside the SAME container command. The cipher file is inside the pod, not in the workstation /etc directory. A successful file test does not automatically export its contents.

403 from GCS: confirm bucket ownership/project, service account email, objectAdmin binding and the actual mounted JSON key. Keep the cipher and identity separate; changing the cipher does not fix IAM.

Unable to find primary: rediscover the primary, execute as postgres, and verify local socket connectivity and pg1-path. Replica execution is not the only possible cause; PGDATA, socket paths and permissions can also be wrong.

WAL archive timeout or /bin/true error: check SHOW archive_command, SHOW archive_library and pending_restart. Confirm the exact config and stanza on all members. Run the full kubectl exec pgBackRest check command from Lab 4 after correcting them. Do not mask the error with archive-check=n or a no-op archive_command.

CR updated but old archive_command remains: compare saved CR parameters, patronictl show-config and pg_settings source/pending_restart. Inspect operator logs. On a confirmed version where the operator does not propagate the desired parameter, use patronictl edit-config to set the same archive_mode and archive_command in postgresql.parameters, then verify actual settings. Keep the CR and dynamic configuration aligned; do not use ALTER SYSTEM to fight Patroni.

```
kubectl -n "$NS" exec -it "$PRIMARY" -c postgres -- \
patronictl edit-config "$CLUSTER"
```

Invalid JSON, line 20 or unexpected YAML field: validate the exact saved file using `python3 -m json.tool` for JSON and `kubectl apply --dry-run=server` for manifests. PVC, Pod, Service and ConfigMap use `apiVersion v1`; StatefulSet uses `apps/v1`; Zalando postgresql uses `acid.zalan.do/v1`. Do not insert Markdown fences into files.

Old volume still referenced: inspect both saved manifest and actual Pod `.spec.volumes`. Update the actual source manifest, recreate only the standalone recovery Pod, and verify `claimName`. Patching a live operator-owned pod is not durable. Do not delete a live PVC to change a mount.

Permission denied on `/restore`: check id, directory ownership and the actual mounted claim. Use the discovered UID/GID, not an assumed 999 or 10001. `fsGroup` assists volume group access only when supported by the storage driver. Use the controlled init-container repair on the new recovery claim if needed.

Restore directory not empty: stop PostgreSQL and use a new recovery PVC/name for another drill, or deliberately move the previous recovery directories aside with adequate capacity. Never use `--delta` or `rm -rf` on an unknown path to bypass the error. The main procedure avoids this by using distinct empty recovery claims.

PostgreSQL cannot start after physical restore: inspect `/restore/postgresql.log`; confirm PostgreSQL major version 18, compatible image/extensions, ownership and recovery configuration. Missing source paths or preload libraries require adjusting the isolated configuration. Do not delete `recovery.signal` to suppress WAL recovery errors.

No `pg_hba.conf` entry: distinguish ordinary SQL from physical replication rules and SSL requirements. Use the operator's standby Secret for `pg_basebackup`. Never add host replication all 0.0.0.0/0 trust.

Suggested schedule and measurement

A starting policy is weekly full backups, daily differential backups, incremental backups every four to six hours, continuous WAL archiving and periodic logical dumps. Choose the actual schedule from measured recovery time, storage cost and accepted data loss. Avoid overlapping backup jobs; a scheduler must discover the current primary and use the same credentials/configuration as this manual.

Record UTC incident time, requested recovery target, last available WAL, restore start, PostgreSQL-ready time, validation completion and application-ready time. RTO is measured to validated service restoration, not merely the end of `pgBackRest` restore. Run restore drills after image, PostgreSQL, storage, IAM or operator changes.

Cleanup after evidence has been saved

Stop recovery PostgreSQL first. If it is already stopped, inspect `pg_ctl` status rather than rerunning startup preparation.

```
for pod in day4-full-recovery day4-pitr-recovery; do
  kubectl -n "$NS" exec -i "$pod" -c postgres -- \
```

```

"$PG_BINDIR/pg_ctl" \
-D /restore/pgdata status || true
done
kubectl -n "$NS" exec -i day4-pitr-recovery -c postgres -- \
"$PG_BINDIR/pg_ctl" \
-D /restore/pgdata -m fast -w stop
kubectl -n "$NS" delete pod \
day4-full-recovery day4-pitr-recovery --ignore-not-found

```

The next command permanently removes the two recovered copies. Execute only after completing validation and retaining evidence.

```

kubectl -n "$NS" delete pvc \
day4-full-recovery-data day4-pitr-recovery-data

```

For optional logical test databases, inspect the names recorded in `evidence/logical-test-db.txt` and `evidence/pitr-copy-db.txt` before dropping them. Do not drop banking or application schemas as generic cleanup. Keep the live pgBackRest ConfigMap, Secrets, image and GCS repository while `archive_command` depends on them. Removing these resources first breaks live WAL archiving.

Completion checklist

Record PASS, FAIL or NOT RUN for each item, with evidence filenames.

- One primary and healthy streaming replicas were identified.
- Logical dump and isolated logical restore were verified.
- `pg_basebackup` used the correct Service, replication user and SSL mode.
- Existing WAL archive settings were recorded before migration.
- Image build/push/pull steps were completed or explicitly skipped as already satisfied.
- GCS bucket, service account and Secret were verified.
- Original repository cipher was preserved and recoverable off-cluster.
- All three mounts are readable by postgres on every member.
- Config path and stanza match the live `archive_command`.
- `stanza-create`, check and actual WAL archiving succeeded.
- Full, incremental and differential backups completed.
- Retention preview was reviewed.
- Physical restore used only new recovery storage.
- Recovered PostgreSQL started and the data was verified.
- PITR retained the good row and excluded the later DELETE.
- Recovery logs and counts support the stated recovery target.
- Cleanup touched only explicitly named recovery/test resources.

Appendix A Disposable backup lab

This independent local-repository exercise preserves the original Kubernetes manual's second path. It is optional after the main GCS drill. Its stanza is lab and its repository is /repo, so its files must not be mixed with stanza postgres-ha or the live GCS prefix.

The example reuses the already verified PostgreSQL 18 plus pgBackRest image from Lab 8, with Spilo/Patroni startup bypassed. This avoids installing packages every time a container starts. It requires REC_IMAGE, PG_BINDIR, PG_UID, PG_GID, REC_SC and POD_SA from 8.1. The data PVC is 5Gi and repository PVC is 10Gi for this tiny exercise. Change both sizes before applying for larger data. Local repository storage demonstrates mechanics, not off-site disaster protection.

A1 Create the password and configuration

```
kubectl -n "$NS" get secret backup-lab-auth
```

If absent, create once:

```
kubectl -n "$NS" create secret generic backup-lab-auth \
  --from-literal=password="$(openssl rand -hex 24)"
```

Create the independent pgBackRest config. Encryption remains covered by the main GCS procedure; this disposable repository uses gzip without encryption.

```
cat > backup-lab-pgbackrest.conf <<'EOF'
[lab]
pg1-path=/lab/pgdata
pg1-port=5432
pg1-socket-path=/tmp
pg1-user=postgres

[global]
repol-path=/repo
repol-retention-full=2
compress-type=gz
compress-level=3
log-level-console=info
log-level-file=off
start-fast=y
EOF
kubectl -n "$NS" create configmap backup-lab-config \
  --from-file=pgbackrest.conf=backup-lab-pgbackrest.conf \
  --dry-run=client -o yaml | kubectl apply -f -
```

A2 Create the complete standalone StatefulSet and storage

The script emits Kubernetes JSON, which kubectl accepts like YAML. It includes the headless Service, both PVCs, the ownership init container, and every mount. This StatefulSet is not controlled by the Zalando operator, so scaling it to zero is not reversed by that operator.

```

cat > make-backup-lab.py <<'PY'
import json, os
E=os.environ
ns=E['NS']; uid=int(E['PG_UID']); gid=int(E['PG_GID'])
image=E['REC_IMAGE']; bindir=E['PG_BINDIR']
def pvc(name, size):
    return {'apiVersion':'v1','kind':'PersistentVolumeClaim',
            'metadata':{'name':name,'namespace':ns},
            'spec':{'accessModes':['ReadWriteOnce'],
                    'storageClassName':E['REC_SC'],
                    'resources':{'requests':{'storage':size}}}}
mounts=[{'name':'data','mountPath':'/lab'},
         {'name':'repo','mountPath':'/repo'},
         {'name':'tmp','mountPath':'/tmp'},
         {'name':'config','mountPath':'/etc/pgbackrest',
          'readOnly':True},
         {'name':'auth','mountPath':'/auth','readOnly':True}]
volumes=[
    {'name':'data','persistentVolumeClaim':{'claimName':'backup-lab-data'}},
    {'name':'repo','persistentVolumeClaim':{'claimName':'backup-lab-repo'}},
    {'name':'tmp','emptyDir':{}},
    {'name':'config','configMap':{'name':'backup-lab-config'}},
    {'name':'auth','secret':{'secretName':'backup-lab-auth'}}]
startup=''
set -eu
if [ ! -s /lab/pgdata/PG_VERSION ]; then
    "$PG_BINDIR/initdb" -D /lab/pgdata -U postgres \\\
    --auth-local=trust --auth-host=scram-sha-256 \\\
    --pwfile=/auth/password
fi
exec "$PG_BINDIR/postgres" -D /lab/pgdata \\\
-c listen_addresses=127.0.0.1 \\\
-c unix_socket_directories=/tmp \\\
-c archive_mode=on \\\
-c "archive_command=pgbackrest --stanza=lab archive-push %p"
...

spec={'serviceAccountName':E['POD_SA'],
      'automountServiceAccountToken':False,
      'securityContext':{'runAsUser':uid,'runAsGroup':gid,
                          'fsGroup':gid},
      'initContainers':[{'name':'permissions','image':image,
                          'command':['sh','-ec',
                                    'mkdir -p /lab/pgdata; '
                                    f'chown -R {uid}:{gid} /lab /repo; '
                                    'chmod 700 /lab/pgdata'],
                          'securityContext':{'runAsUser':0,'runAsGroup':0},
                          'volumeMounts':mounts[:2]}],
      'containers':[{'name':'postgres','image':image,

```

```

    'command':['sh','-ec',startup],
    'env':[{'name':'PG_BINDIR','value':bindir}],
    'volumeMounts':mounts,
    'readinessProbe':{'exec':{'command':[
        bindir+'/pg_isready','-h','/tmp','-U','postgres']},
        'initialDelaySeconds':5,'periodSeconds':5}},
    'volumes':volumes}
src=json.load(open('evidence/recovery-source-pod.json'))
if src['spec'].get('imagePullSecrets'):
    spec['imagePullSecrets']=src['spec']['imagePullSecrets']
sts={'apiVersion':'apps/v1','kind':'StatefulSet',
    'metadata':{'name':'backup-lab','namespace':ns},
    'spec':{'serviceName':'backup-lab','replicas':1,
        'selector':{'matchLabels':{'app':'backup-lab'}},
        'template':{'metadata':{'labels':{'app':'backup-lab'}},
            'spec':spec}}}}
svc={'apiVersion':'v1','kind':'Service',
    'metadata':{'name':'backup-lab','namespace':ns},
    'spec':{'clusterIP':'None','selector':{'app':'backup-lab'},
        'ports':[{'name':'postgres','port':5432,'targetPort':5432}]}}
items=[pvc('backup-lab-data','5Gi'),
    pvc('backup-lab-repo','10Gi'),svc,sts]
json.dump({'apiVersion':'v1','kind':'List','items':items},
    open('manifests/backup-lab.json','w'),indent=2)
# Recovery mounts the same disposable PVCs only after scale-down.
rec=json.loads(json.dumps(spec))
rec.pop('initContainers',None)
rec['restartPolicy']='Never'
rec['containers'][0]['command']=['sh','-ec','sleep infinity']
rec['containers'][0].pop('readinessProbe',None)
recovery={'apiVersion':'v1','kind':'Pod',
    'metadata':{'name':'backup-lab-recovery','namespace':ns},
    'spec':rec}
json.dump(recovery,open('manifests/backup-lab-recovery.json','w'),
    indent=2)
PY
python3 make-backup-lab.py
kubectl apply --dry-run=server -f manifests/backup-lab.json
kubectl apply -f manifests/backup-lab.json
kubectl -n "$NS" rollout status statefulset/backup-lab \
    --timeout=10m
kubectl -n "$NS" logs backup-lab-0 -c postgres --tail=40

```

Expected: standalone PostgreSQL starts. Until stanza-create succeeds, archive errors can appear because this new repository has no stanza. The headless Service is included for StatefulSet identity; the database listens on loopback only in this drill.

A3 Stanza and local data

```
kubectl -n "$NS" exec -i backup-lab-0 -c postgres -- \
  pgbackrest --stanza=lab stanza-create
kubectl -n "$NS" exec -i backup-lab-0 -c postgres -- \
  pgbackrest --stanza=lab check
kubectl -n "$NS" exec -i backup-lab-0 -c postgres -- \
  "$PG_BINDIR/createdb" -h /tmp \
  -U postgres drlab
kubectl -n "$NS" exec -i backup-lab-0 -c postgres -- \
  "$PG_BINDIR/psql" -X -h /tmp \
  -U postgres -d drlab <<'SQL'
\set ON_ERROR_STOP on
CREATE TABLE orders(id int PRIMARY KEY, note text);
INSERT INTO orders VALUES (1, 'full');
SELECT pg_switch_wal();
TABLE orders;
SQL
```

Expected: orders contains row 1; stanza and archive checks succeed. On reruns, inspect existing objects rather than blindly recreating the database or duplicate primary keys.

A4 Logical backup and restore in the disposable cluster

```
kubectl -n "$NS" exec -i backup-lab-0 -c postgres -- \
  "$PG_BINDIR/pg_dump" -h /tmp \
  -U postgres -Fc -d drlab > backups/logical/drlab.dump
kubectl -n "$NS" exec -i backup-lab-0 -c postgres -- \
  "$PG_BINDIR/createdb" -h /tmp \
  -U postgres drlab_copy
cat backups/logical/drlab.dump \
| kubectl -n "$NS" exec -i backup-lab-0 -c postgres -- \
  \
  "$PG_BINDIR/pg_restore" -h /tmp -U postgres \
  -d drlab_copy --exit-on-error
kubectl -n "$NS" exec -i backup-lab-0 -c postgres -- \
  "$PG_BINDIR/psql" -X -h /tmp \
  -U postgres -d drlab_copy -c 'TABLE orders;'
```

Expected: drlab_copy has row 1.

A5 Full incremental and differential backups

```
kubectl -n "$NS" exec -i backup-lab-0 -c postgres -- \
  pgbackrest --stanza=lab --type=full backup
kubectl -n "$NS" exec -i backup-lab-0 -c postgres -- \
  "$PG_BINDIR/psql" -X -h /tmp \
  -U postgres -d drlab -v ON_ERROR_STOP=1 -c \
  "INSERT INTO orders VALUES (2, 'incremental');"
kubectl -n "$NS" exec -i backup-lab-0 -c postgres -- \
  pgbackrest --stanza=lab --type=incr backup
kubectl -n "$NS" exec -i backup-lab-0 -c postgres -- \
  "$PG_BINDIR/psql" -X -h /tmp \
  -U postgres -d drlab -v ON_ERROR_STOP=1 -c \
  "INSERT INTO orders VALUES (3, 'differential');"
kubectl -n "$NS" exec -i backup-lab-0 -c postgres -- \
  pgbackrest --stanza=lab --type=diff backup
kubectl -n "$NS" exec -i backup-lab-0 -c postgres -- \
  pgbackrest --stanza=lab info
```

Expected: F, I and D labels. The chosen local repository remains separate from GCS and the live-cluster stanza.

A6 Add WAL data and stop before restore

```
kubectl -n "$NS" exec -i backup-lab-0 -c postgres -- \
  "$PG_BINDIR/psql" -X -h /tmp \
  -U postgres -d drlab -v ON_ERROR_STOP=1 -c \
  "INSERT INTO orders VALUES (4, 'archived WAL');"
kubectl -n "$NS" exec -i backup-lab-0 -c postgres -- \
  "$PG_BINDIR/psql" -X -h /tmp \
  -U postgres -d postgres -c 'SELECT pg_switch_wal();'
kubectl -n "$NS" exec -i backup-lab-0 -c postgres -- \
  pgbackrest --stanza=lab check
kubectl -n "$NS" scale statefulset/backup-lab --replicas=0
kubectl -n "$NS" wait --for=delete pod/backup-lab-0 \
  --timeout=180s
kubectl apply -f manifests/backup-lab-recovery.json
kubectl -n "$NS" wait --for=condition=Ready \
  pod/backup-lab-recovery --timeout=10m
```

Scale-down stops the disposable PostgreSQL container. Do not start a recovery pod until the source pod is completely deleted. If shutdown was not clean, inspect the log and storage before restoring; do not ignore a postmaster.pid check failure. The recovery pod mounts only backup-lab-data and backup-lab-repo, never acid-postgres-ha data.

A7 Restore and restart the disposable database

```
kubectl -n "$NS" exec -i backup-lab-recovery -c postgres -- \
  sh -ec '
  test ! -e /lab/pgdata/postmaster.pid
  mv /lab/pgdata "/lab/pgdata.before-restore-$(date +%s)"
  mkdir -m 700 /lab/pgdata
  pgbackrest --stanza=lab --type=default restore
'

kubectl -n "$NS" delete pod backup-lab-recovery --wait=true
kubectl -n "$NS" scale statefulset/backup-lab --replicas=1
kubectl -n "$NS" rollout status statefulset/backup-lab \
  --timeout=10m
kubectl -n "$NS" exec -i backup-lab-0 -c postgres -- \
  "$PG_BINDIR/psql" -X -h /tmp \
  -U postgres -d drlab -c 'TABLE orders;'
kubectl -n "$NS" exec -i backup-lab-0 -c postgres -- \
  "$PG_BINDIR/psql" -X -h /tmp \
  -U postgres -d postgres -c 'SELECT pg_is_in_recovery();'
```

Expected: rows 1 through 4 and recovery false. Default archive recovery replays available WAL rather than stopping at the differential backup's earliest consistent point. The disposable recovered cluster can archive its new timeline to its own disposable repository. Keep enough PVC space for the moved-aside copy; do not delete it until the restore is verified.

A8 Disposable PITR

```
kubectl -n "$NS" exec -i backup-lab-0 -c postgres -- \
  pgbackrest --stanza=lab --type=full backup
kubectl -n "$NS" exec -i backup-lab-0 -c postgres -- \
  pgbackrest --stanza=lab --output=json info \
  > evidence/disposable-pitr-info.json
LAB_FULL=$(jq -er '.[0].backup
| map(select(.type == "full")) | last | .label' \
evidence/disposable-pitr-info.json)
kubectl -n "$NS" exec -i backup-lab-0 -c postgres -- \
  "$PG_BINDIR/psql" -X -h /tmp \
  -U postgres -d drlab -v ON_ERROR_STOP=1 -c \
  "INSERT INTO orders VALUES (5, 'good row');"
LAB_TARGET=$(kubectl -n "$NS" exec -i backup-lab-0 -c postgres -- \
  "$PG_BINDIR/psql" \
  -X -h /tmp -U postgres -d postgres -Atqc \
  "SELECT to_char(clock_timestamp() AT TIME ZONE 'UTC',
    'YYYY-MM-DD HH24:MI:SS.US') || '+00';")
printf '%s\n' "$LAB_TARGET" > evidence/disposable-target.txt
sleep 3
kubectl -n "$NS" exec -i backup-lab-0 -c postgres -- \
  "$PG_BINDIR/psql" -X -h /tmp \
  -U postgres -d drlab -v ON_ERROR_STOP=1 -c \
  'DELETE FROM orders; SELECT count(*) FROM orders;'
kubectl -n "$NS" exec -i backup-lab-0 -c postgres -- \
```

```

"$PG_BINDIR/psql" -X -h /tmp \
-U postgres -d postgres -c 'SELECT pg_switch_wal();'
kubectl -n "$NS" exec -i backup-lab-0 -c postgres -- \
  pgbackrest --stanza=lab check
kubectl -n "$NS" scale statefulset/backup-lab --replicas=0
kubectl -n "$NS" wait --for=delete pod/backup-lab-0 \
  --timeout=180s
kubectl apply -f manifests/backup-lab-recovery.json
kubectl -n "$NS" wait --for=condition=Ready \
  pod/backup-lab-recovery --timeout=10m
kubectl -n "$NS" exec -i backup-lab-recovery -c postgres -- \
  env LAB_FULL="$LAB_FULL" \
  LAB_TARGET="$LAB_TARGET" sh -ec '
test ! -e /lab/pgdata/postmaster.pid
mv /lab/pgdata "/lab/pgdata.before-pitr-$(date +%s)"
mkdir -m 700 /lab/pgdata
pgbackrest --stanza=lab --set="$LAB_FULL" \
  --type=time --target="$LAB_TARGET" \
  --target-action=promote restore
'
kubectl -n "$NS" delete pod backup-lab-recovery --wait=true
kubectl -n "$NS" scale statefulset/backup-lab --replicas=1
kubectl -n "$NS" rollout status statefulset/backup-lab \
  --timeout=10m
kubectl -n "$NS" exec -i backup-lab-0 -c postgres -- \
  "$PG_BINDIR/psql" -X -h /tmp \
  -U postgres -d drlab -c 'TABLE orders;'
kubectl -n "$NS" exec -i backup-lab-0 -c postgres -- \
  "$PG_BINDIR/psql" -X -h /tmp \
  -U postgres -d postgres -c \
  'SELECT pg_is_in_recovery();
  SELECT timeline_id FROM pg_control_checkpoint();'

```

Expected: rows 1 through 5, the DELETE excluded, and a promoted timeline. Inspect kubectl logs backup-lab-0 -c postgres for the recovery target messages. Do not accept only a Ready pod as proof of the correct recovered data.

A9 Retention and cleanup

```
kubectl -n "$NS" exec -i backup-lab-0 -c postgres -- \
  pgbackrest --stanza=lab --dry-run expire
kubectl -n "$NS" exec -i backup-lab-0 -c postgres -- \
  pgbackrest --stanza=lab info
kubectl -n "$NS" scale statefulset/backup-lab --replicas=0
kubectl -n "$NS" wait --for=delete pod/backup-lab-0 \
  --timeout=180s
kubectl -n "$NS" delete statefulset backup-lab
kubectl -n "$NS" delete service backup-lab
kubectl -n "$NS" delete pod backup-lab-recovery \
  --ignore-not-found
kubectl -n "$NS" delete configmap backup-lab-config
kubectl -n "$NS" delete secret backup-lab-auth
```

Only when you no longer need the disposable data and local backups:

```
kubectl -n "$NS" delete pvc backup-lab-data backup-lab-repo
```

This local-repository cleanup does not remove live-cluster backups, live Secrets or the GCS bucket.

A10 Separate operator cluster for logical restore

The original manual also created acid-recovery-test for logical restoration. Retain this alternative when a separate operator-managed logical target is desired. Use the same tested PostgreSQL 18 image; the CR starts a NEW cluster with its own storage and Patroni identity. This is not the manual physical restore workflow from Lab 8.

```
cat > manifests/acid-recovery-test.yaml <<EOF
apiVersion: acid.zalan.do/v1
kind: postgresql
metadata:
  name: acid-recovery-test
  namespace: $NS
spec:
  teamId: acid
  dockerImage: $REC_IMAGE
  numberOfInstances: 1
  volume:
    size: 10Gi
    storageClass: $REC_SC
  postgresql:
    version: "18"
EOF
kubectl apply --dry-run=server \
  -f manifests/acid-recovery-test.yaml
kubectl apply -f manifests/acid-recovery-test.yaml
kubectl -n "$NS" get pods -w
```

Wait for acid-recovery-test-0 to be Ready, then Ctrl+C. A private image requires the operator's pod ServiceAccount/pull credentials to be configured as in 4.5.

```
LOGICAL_REC=acid-recovery-test-0
kubectl -n "$NS" exec -i "$LOGICAL_REC" -c postgres -- \
  runuser -u postgres -- createdb -U postgres recoverydb
cat "$BACKUP_FILE" | kubectl -n "$NS" exec -i "$LOGICAL_REC" -c postgres -- \
  runuser -u postgres -- pg_restore \
  -U postgres -d recoverydb --no-owner --no-privileges \
  --no-tablespaces --exit-on-error
kubectl -n "$NS" exec -i "$LOGICAL_REC" -c postgres -- \
  runuser -u postgres -- psql -X -U postgres -d recoverydb \
  -c 'TABLE day4_lab.events;'
```

If BACKUP_FILE was lost between terminals, set it to the exact banking custom dump from Lab 1 after listing backups/logical; do not pass a wildcard matching multiple files to kubectl cp or cat.

After validation, delete only the test CR and inspect its retained PVC before choosing whether to delete the test storage:

```
kubectl -n "$NS" delete postgresql acid-recovery-test
kubectl -n "$NS" get pvc
```

Do not use a broad label deletion that could select the live cluster.

Appendix B Coverage and corrections

The source versions were compared for unique procedures, not merely concatenated. Duplicate steps were consolidated and obsolete commands replaced by explicit procedures. Lab numbering remains 0 through 9; setup details now sit in the lab that needs them.

Source reconciliation

Kubernetes_PostgreSQL18_Backup_Recovery_Lab_Manual(1).pdf: retained live inspection, logical dumps, separate operator logical target and disposable local-repository exercises. Added complete storage and Service creation, immutable-image reuse, PostgreSQL startup and explicit verification. Replaced the unverified native force-backup annotation with an explanation of its limitation.

DAY4_Postgres_HA_Patroni_Live_Backup_DR_Lab_Manual_CORRECTED_6(1).pdf: retained original coverage through Lab 9, image creation, repository encryption, additional mounts, failover, monitoring and RPO/RTO. Replaced registry.example.com with concrete registry creation and image tagging steps. Preserved the base image USER instead of forcing USER postgres.

DAY4_Postgres_HA_Patroni_Live_Backup_DR_Lab_Manual_CORRECTED_7(1).pdf: retained Docker Kubernetes GCS service-account authentication and mounted cipher handling. Added bucket creation and explicit new-versus-existing Secret branches.

DAY4_Postgres_HA_Patroni_Live_Backup_DR_Lab_Manual_CORRECTED_8(1).pdf: retained JSON patches, mount validation, primary-aware stanza creation and archive-command repair. Mount merging now preserves unrelated additionalVolumes.

DAY4_Postgres_HA_Patroni_Live_Backup_DR_Lab_Manual_CORRECTED_9_SINGLE_LINE_COMMA NDS(2).pdf: retained its procedural coverage while replacing clipped long lines with properly continued Bash commands and complete heredocs. Key-creation policy failures now require an approved administrative resolution rather than routine policy disabling.

DAY4_Postgres_HA_Patroni_Live_Backup_DR_Lab_Manual_CORRECTED_11_DRYRUN_FINAL(1).pdf: retained the explicit config path, stanza consistency and WAL verification corrections. Restored the omitted image/setup and recovery implementation details.

DAY4_Postgres_HA_Patroni_Live_Backup_DR_Lab_Manual_READABLE_COPYPASTE_FINAL(1).pdf: retained the later dry-run fixes but rebuilt the layout at readable body/code sizes. Replaced restore-toolbox and TARGET_TIME placeholders with complete creation, recording, startup and verification steps.

Explicit corrections to earlier behavior

Labs 0 through 3 retain their intended exercises, but visibly clipped commands have been repaired rather than reproduced. Source tools are run through explicit kubectl exec commands with the required Linux postgres-user prefix and SSL is explicit for network replication.

The earlier pattern of creating an operator-owned recovery cluster, scaling its StatefulSet to zero and manually writing into its PVC is replaced in the physical drills by standalone recovery pods. This avoids operator reconciliation restarting PostgreSQL during restore or rejecting mismatched Patroni identity.

The original full-backup exercise created its demonstration table only after the backup. This edition seeds data before the full baseline and labels subsequent inserts so expected recovery contents are unambiguous.

The pgBackRest check runs after a valid archive command is active. It is not used as a prerequisite before the new archive pipeline exists. Successful `/bin/true` archiving is never accepted as recovery evidence.

A recovery server exports the cipher environment for its generated `restore_command`. Supplying the passphrase only to the one-time restore process is insufficient when PostgreSQL later needs `archive-get` during WAL replay.

Recovery targets use discovered image UID/GID, complete mounts and remapped tablespaces. They start PostgreSQL explicitly and verify both data and recovery state. There are no `RECOVERY_TOOLBOX` or `FULL_LABEL_HERE` placeholders left for these steps.

References and verification scope

Official documentation consulted for the consolidated workflow. Links are provided for installed-version details and administrator review; commands in this manual are an adaptation to the supplied lab environment.

PostgreSQL 18 continuous archiving and PITR

<https://www.postgresql.org/docs/18/continuous-archiving.html>

PostgreSQL 18 pg_basebackup

<https://www.postgresql.org/docs/18/app-pgbasebackup.html>

PostgreSQL 18 recovery target settings

<https://www.postgresql.org/docs/18/runtime-config-wal.html>

pgBackRest command reference and configuration reference

<https://pgbackrest.org/command.html>

<https://pgbackrest.org/configuration.html>

pgBackRest releases and PostgreSQL compatibility

<https://pgbackrest.org/release.html>

Zalando operator cluster manifest and administrator guide

https://postgres-operator.readthedocs.io/en/latest/reference/cluster_manifest/

<https://postgres-operator.readthedocs.io/en/latest/administrator/>

Kubernetes security context and volume ownership

<https://kubernetes.io/docs/tasks/configure-pod-container/security-context/>

Google Artifact Registry authentication and image publishing

<https://docs.cloud.google.com/artifact-registry/docs/docker/authentication>

<https://docs.cloud.google.com/artifact-registry/docs/docker/pushing-and-pulling>

Google Cloud Storage bucket creation

<https://docs.cloud.google.com/storage/docs/creating-buckets>

Validation scope: document commands were reviewed for ordering, variable dependencies, shell syntax and manifest construction. The manual was rendered and checked for readable, selectable text. Live execution against your Kubernetes/GCP environment remains necessary; operator version, image contents, permissions, installed extensions and storage-driver behavior can affect results.